



# Reliability in Event-Driven Design

## How Correctness Boundaries Shape Process-Handling Choices and Recovery Behaviour

Herman Karlsson

This thesis is submitted to the Faculty of Computer Science at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Higher Education Diploma in Software Engineering. The thesis is equivalent to 10 weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

**Contact Information:**

Author:

Herman Karlsson

E-mail: hekr23@student.bth.se

University advisor:

Ramin Irani

Department of Computer Science

Faculty of Computer Science  
Blekinge Institute of Technology  
SE-371 79 Karlskrona, Sweden

Internet : [www.bth.se](http://www.bth.se)  
Phone : +46 455 38 50 00  
Fax : +46 455 38 50 57

---

## Acknowledgments

I thank my supervisor, Ramin Irani, for his discerning feedback at key moments in the thesis process, particularly during the proposal and final submission phases.

---

# Abstract

**Background.** Reliability in event-driven processes is difficult to judge when the conditions that make a process acceptable are left implicit. An event-driven process may preserve work, delay it, replay it, duplicate it, or omit it, but these behaviours do not have a fixed process-level meaning. Replaying a retained event may appear to improve reliability, yet no longer protect correctness once the relevant deadline has passed. A missed update may no longer matter. Deferred settlement may reduce duplicate harm while moving burden into reconciliation. The value of a handling choice therefore depends on the process requirement it is meant to satisfy.

**Objectives.** This thesis develops a boundary-first framework for reasoning about reliability in event-driven processes. The aim is to make explicit what a process must protect before judging which process-handling model is appropriate.

**Methods.** The thesis follows a design science research approach. It develops the framework and examines it through a controlled, containerised empirical suite using transient and retained event handling. The suite compares deadline-constrained, state-oriented, and required-effect scenarios across three process-handling models: transient/immediate, retained/immediate, and retained/deferred. Each scenario cell combines a protected condition, a process-handling model, and a defined disturbance, so that selected differences in preservation, replay, repayment, and settlement can be observed across the suite. Repeated executions check that the main comparative findings do not depend on a single execution trace. The empirical comparison is exploratory and focuses on process-handling behaviour under disturbance.

**Results.** Across this controlled suite, the comparison shows that the value of replay, retention, immediate handling, and deferred settlement depends on the protected condition at stake. For deadline-constrained work, recovery matters only while unsettled work can still be repaid before expiry. For state-oriented work, replay matters when missed state remains relevant, while later state may overtake earlier misses. For required-effect work, replay can recover downstream handling gaps after emission, but not source omission. Deferred settlement can reduce immediate duplicate pressure, but it does so by moving part of the burden into later reconciliation.

**Conclusions.** The thesis contributes a process-level framework for comparing handling choices under explicit correctness requirements. Its value lies in making the comparison inspectable: what the process must protect, what can threaten that requirement, which process-handling model is chosen, and what trade-off follows. The guide gives this reasoning a more concrete form, so that the design judgments behind handling choices can be understood more clearly.

**Keywords:** event-driven processes, reliability, design science research, correctness boundaries, process-handling models

---

# Contents

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Acknowledgments</b>                                        | <b>i</b>  |
| <b>Abstract</b>                                               | <b>ii</b> |
| <b>1 Introduction</b>                                         | <b>1</b>  |
| 1.1 Background and motivation . . . . .                       | 1         |
| 1.1.1 A simple example . . . . .                              | 1         |
| 1.1.2 Where the difficulty appears . . . . .                  | 2         |
| 1.2 Core conceptual basis . . . . .                           | 2         |
| 1.2.1 Correctness boundary . . . . .                          | 2         |
| 1.2.2 Protected conditions . . . . .                          | 2         |
| 1.2.3 Process-handling models . . . . .                       | 3         |
| 1.2.4 How the concepts are used . . . . .                     | 3         |
| 1.3 Research questions . . . . .                              | 4         |
| 1.4 Aim of the thesis . . . . .                               | 4         |
| 1.5 Outline of the thesis . . . . .                           | 4         |
| <b>2 Related Work</b>                                         | <b>5</b>  |
| 2.1 Event-driven work as process-level obligation . . . . .   | 6         |
| 2.2 Mechanisms and the conditions they serve . . . . .        | 6         |
| 2.3 Three kinds of protected conditions in context . . . . .  | 7         |
| 2.3.1 Timing and validity windows . . . . .                   | 7         |
| 2.3.2 State divergence and acceptable state . . . . .         | 7         |
| 2.3.3 Downstream effects and duplicate control . . . . .      | 7         |
| 2.4 Summary and relation to this thesis . . . . .             | 8         |
| <b>3 Method</b>                                               | <b>9</b>  |
| 3.1 Research design . . . . .                                 | 9         |
| 3.1.1 Design-science framing . . . . .                        | 9         |
| 3.1.2 Controlled comparison and disturbance . . . . .         | 10        |
| 3.2 Implementation environment and platform choices . . . . . | 10        |
| 3.2.1 Runtime arrangement . . . . .                           | 10        |
| 3.2.2 Redis transport choices . . . . .                       | 11        |
| 3.2.3 Services, reporting, and versions . . . . .             | 11        |
| 3.3 Scenario and run structure . . . . .                      | 11        |
| 3.3.1 Condition modules and disturbance profiles . . . . .    | 11        |
| 3.3.2 Scenario-cell construction . . . . .                    | 12        |
| 3.3.3 Repetitions and stability . . . . .                     | 12        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 3.4      | Operationalising the protected conditions . . . . .                   | 13        |
| 3.5      | Metrics and interpretation . . . . .                                  | 14        |
| 3.6      | Reporting discipline and scope . . . . .                              | 15        |
| <b>4</b> | <b>Results and Analysis</b>                                           | <b>17</b> |
| 4.1      | Deadline-constrained results . . . . .                                | 17        |
| 4.2      | State-oriented results . . . . .                                      | 19        |
| 4.3      | Required-effect results . . . . .                                     | 20        |
| 4.4      | Remaining possibility after disturbance . . . . .                     | 22        |
| <b>5</b> | <b>Discussion</b>                                                     | <b>24</b> |
| 5.1      | Answer to RQ1: protected conditions and process acceptability . . . . | 24        |
| 5.2      | Answer to RQ2: handling models under controlled disturbance . . . .   | 25        |
| 5.3      | Answer to RQ3: correctness boundaries and handling choice . . . . .   | 26        |
| 5.3.1    | Boundary-first design guide . . . . .                                 | 26        |
| 5.3.2    | Producer omission and producer-integrity controls . . . . .           | 27        |
| 5.3.3    | The cancellation case as a process chain . . . . .                    | 28        |
| 5.3.4    | Connected conditions and boundary coherence . . . . .                 | 29        |
| 5.3.5    | Further examples . . . . .                                            | 30        |
| 5.3.6    | Food delivery service . . . . .                                       | 30        |
| 5.3.7    | E-commerce checkout and order processing . . . . .                    | 30        |
| 5.3.8    | Scope and realism . . . . .                                           | 31        |
| 5.4      | Limitations and claim boundaries . . . . .                            | 31        |
| 5.4.1    | Framework scope . . . . .                                             | 31        |
| 5.4.2    | Empirical scope . . . . .                                             | 31        |
| 5.4.3    | Guide and architectural scope . . . . .                               | 32        |
| 5.5      | Ethical and professional considerations . . . . .                     | 32        |
| <b>6</b> | <b>Conclusions and Future Work</b>                                    | <b>34</b> |
| 6.1      | Future Work . . . . .                                                 | 34        |
| 6.1.1    | Design practice and guide use . . . . .                               | 34        |
| 6.1.2    | Platform mechanisms and operational judgment . . . . .                | 34        |
| 6.1.3    | Connected conditions and framework extension . . . . .                | 35        |
| 6.2      | Findings and contribution in context . . . . .                        | 35        |
| 6.2.1    | From mechanism to process judgment . . . . .                          | 35        |
| 6.2.2    | What the evidence is evidence of . . . . .                            | 35        |
| 6.2.3    | Contribution and claim boundary . . . . .                             | 36        |
| 6.3      | Conclusion . . . . .                                                  | 37        |
|          | <b>References</b>                                                     | <b>38</b> |
|          | <b>A Boundary-first design guide</b>                                  | <b>41</b> |
|          | <b>B Empirical suite synopsis</b>                                     | <b>42</b> |
|          | <b>Supplementary material</b>                                         | <b>49</b> |

---

## List of Figures

|     |                                                        |    |
|-----|--------------------------------------------------------|----|
| 4.1 | Deadline outcome burden . . . . .                      | 17 |
| 4.2 | Runtime resolution and final deadline burden . . . . . | 18 |
| 4.3 | Cumulative backlog-shock accounting . . . . .          | 19 |
| 4.4 | State-oriented adequacy . . . . .                      | 20 |
| 4.5 | Source omission boundary by received events . . . . .  | 21 |
| 4.6 | Duplicate-pressure containment . . . . .               | 21 |
| B.1 | Empirical suite execution pipeline . . . . .           | 43 |

---

## List of Tables

|     |                                                                    |    |
|-----|--------------------------------------------------------------------|----|
| 3.1 | Design-science role of the framework and empirical suite. . . . .  | 10 |
| 3.2 | Frozen empirical suite manifest. . . . .                           | 13 |
| 3.3 | Operationalisation of protected conditions. . . . .                | 14 |
| 3.4 | Metric groups by protected condition. . . . .                      | 15 |
| 4.1 | Duplicate-containment trade-off under deferred settlement. . . . . | 22 |
| 5.1 | Boundary questions and handling considerations. . . . .            | 27 |
| A.1 | Boundary-first guide for process-handling choice. . . . .          | 41 |
| B.1 | Operational definitions of reported metrics. . . . .               | 45 |
| B.2 | Scenario and parameter specification. . . . .                      | 46 |
| B.3 | Disturbance-to-channel map. . . . .                                | 47 |
| B.4 | Evaluation controls and scope protection. . . . .                  | 48 |

### 1.1 Background and motivation

Event-driven design often carries work across time and across component boundaries. An order, update, payment, booking, notification, or intervention may begin in one part of a system and be completed somewhere else, later, through an event flow. This provides useful flexibility: components can be decoupled, uneven demand can be absorbed, and work can continue asynchronously. It can also make reliability harder to judge, because the process outcome cannot be read from event movement alone.

Events are carried and handled through mechanisms. Some concern how an event is delivered, kept, or made available for later handling. Others belong to the application logic around the event, such as updating stored state, recognising repeated triggers, or correcting an outcome later. These mechanisms may be useful and still leave the relevant process outcome uncertain, because they do not by themselves settle whether the process is doing the work it is meant to do.

#### 1.1.1 A simple example

Consider a local fitness center booking service. A customer cancels a class booking. If the cancellation is accepted, the system has to record that fact so that the booking no longer occupies a place. Other work may then follow: the visible class availability may need to reflect the released place, someone on the waiting list may need to be offered that place before booking closes or the class begins, and a class credit may need to be issued if the cancellation qualifies. From the customer's perspective this may appear to be one cancellation, but the system is handling several concerns at once.

The availability count has to be adjusted so that customers and staff can rely on whether a place is available. If the class has a waiting list, the released place may need to be offered to another customer while there is still time to take it. If the cancellation qualifies, a class credit for a future booking should be issued to the customer. The availability count can mislead, the offer can arrive too late, and the credit can fail to be issued or be issued more than once. The thesis builds from these elemental differences between state, time, and action: what must be true of the relevant state, the window in which completion still matters, and what must be done.

### 1.1.2 Where the difficulty appears

The same difficulty appears in studies of event-driven and microservice systems, where developers report difficulty designing, inspecting, and reasoning about flows once application work crosses service boundaries and depends on events, ordering assumptions, and transactional support [1, 2].

Some of this complication is structural. As work is split across services, carried by events, and completed later through different forms of handling, it can become difficult to see what the design is being asked to protect. A useful response can be to step back and ask, on a process-by-process basis:

*What must hold for this process to reach an acceptable outcome?*

This does not settle the handling choice by itself, but it gives the design reasoning a more stable point of reference. It restores a clearer focus that is easily obscured by business rules, component boundaries, timing assumptions, and the machinery of delivery, recovery, and settlement.

## 1.2 Core conceptual basis

That question is the thesis’s point of departure. The working vocabulary introduced here separates the process-level acceptability judgment, the conditions that define it, and the handling models later compared in the thesis.

### 1.2.1 Correctness boundary

The name used here for this process-level judgment is a *correctness boundary*. A correctness boundary is the line between acceptable and unacceptable case outcomes for a scoped process. The boundary is drawn around the acceptability of the process outcome, not around the component or transport mechanism that carries it. It states what must hold for a case to count as acceptable, and it may be defined by one *protected condition* or by a connected set of protected conditions. The point is to reason about those conditions clearly before the handling choice is judged.

The term thus keeps the process boundary distinct from the technical means used to protect it. Those means include event-transport capabilities such as publication, delivery, retention, and replay, and application-level safeguards such as idempotence and settlement logic. The design question is which handling model is appropriate to the boundary defined by the process’s protected condition or conditions.

### 1.2.2 Protected conditions

To make the comparison concrete, the analysis focuses on three kinds of protected conditions. In *deadline-constrained* work, correctness depends on reaching a valid outcome before a window closes. In *state-oriented* work, correctness depends on whether missed or delayed updates still matter to the state the process is required to preserve or expose. In *required-effect* work, correctness depends on whether one downstream action happens reliably and without harmful duplication. These kinds of conditions are not mutually exclusive types of real-world process. They are analytical

distinctions that help show what defines a process boundary, including where several conditions have to be considered together.

### 1.2.3 Process-handling models

A *process-handling model* is the arrangement by which transport mechanisms and application-side safeguards are combined for an event-driven process in relation to what it must protect. It covers both the movement and availability of emitted work and the safeguards that make later processing safe, recoverable, or open to settlement. In this sense, the model describes how the process is carried toward an acceptable outcome, including when delivery, processing, or settlement is disturbed.

In an event-driven process, a producing component emits work as an event or message, and a consuming component receives that work for processing. The transport part of the choice lies between emission and downstream processing: whether emitted work exists only during an immediate delivery-and-processing opportunity, or remains available when that opportunity does not lead to completion.

This thesis uses three transport-anchored process-handling models, later operationalised in Chapter 3:

*transient/immediate*: emitted work is available only for immediate delivery and processing. If that opportunity does not lead to completion, no retained work remains for replay.

*retained/immediate*: emitted work is retained while immediate processing is attempted. If that attempt does not complete, the retained work remains available for replay.

*retained/deferred*: retained transport is paired with deferred application-side settlement. The emitted work is kept as a durable basis for later settlement rather than being completed at the initial processing point.

The three labels distinguish whether emitted work is transient, retained for replay, or retained as the basis for deferred settlement. They name the base contrast used later in the thesis, while application-side safeguards remain part of the wider process-handling model where they bear on what the process must protect. This keeps the labels clear without losing the process-level question: whether the arrangement protects what makes the process acceptable.

### 1.2.4 How the concepts are used

Together, these distinctions set up the thesis's central comparison. A correctness boundary states what must still be true for a scoped process. A protected condition, or a connected set of protected conditions, defines that boundary by stating what must hold for the process outcome to be acceptable. The handling model describes how the process is arranged in response. The thesis uses this relation to judge handling choices against explicit process-level requirements, and examines the comparison through a controlled empirical suite. The suite narrows the setting so that selected handling differences can be observed clearly under defined disturbances. Its

role is to support the comparison, not to reproduce the full complexity of deployed event-driven systems or mirror their full range of operating conditions.

## 1.3 Research questions

The thesis is guided by three research questions:

**RQ1.** *Which kinds of protected conditions can be used to reason about correctness in the event-driven processes examined here?*

**RQ2.** *How do the selected process-handling models shape preservation, replay, re-payment, and settlement under controlled disturbance?*

**RQ3.** *How can explicit correctness boundaries guide process-handling choices in event-driven design?*

## 1.4 Aim of the thesis

Taken together, these questions set the direction of the thesis. The work proceeds from a simple premise: before an event-driven process can be judged reliable, what it must protect has to be made explicit. The thesis sets out to develop this premise into a process-level framework and to examine it through controlled empirical comparison, using deadline-constrained, state-oriented, and required-effect conditions to compare selected handling models under disturbance. The aim is to make handling choices easier to judge against what makes the process acceptable.

## 1.5 Outline of the thesis

Chapter 2 situates the thesis in relation to the literature most relevant to event-driven reliability, correctness, recovery, and coordination. Chapter 3 explains the research design, the empirical suite, the interpretive metrics, and the study's scope limits. Chapter 4 presents the empirical results across the three kinds of protected conditions. Chapter 5 discusses what those results mean in relation to the research questions, develops the boundary-first guide, and states the thesis's claim boundaries. Chapter 6 identifies future work, places the thesis in context, and closes the argument.

This chapter positions the argument within the bodies of work most directly relevant to it. Chapter 1 introduced the thesis’s boundary-first vocabulary and scoped the study. The task here is to relate that vocabulary to the strands of distributed-systems and event-driven literature that matter most for how correctness, recovery, and process-handling choices are understood.

The study sits between established distributed-systems research and practical event-driven design. Prior work has given rich accounts of publish-subscribe decoupling, end-to-end reasoning, fault recovery, tail behavior, replication, stream processing, and compensating action [3, 4, 5, 6, 7, 8, 9, 10]. These works provide the background for the argument developed here. They show that reliability is never only a matter of moving messages. It is also a matter of what a system is trying to preserve, how that preservation is attempted, and what costs follow from the chosen form of handling.

These bodies of work make the starting point concrete. Event-driven design already offers many ways to publish, retain, replay, process, coordinate, and settle work. What remains difficult is how those means should be judged when a designer must decide what kind of correctness is actually at stake in a concrete flow. This matters all the more as event-streaming and stream-processing infrastructure have become more common and prominent in both research and practice [10]. The more readily such support is available, the easier it becomes to reason from available capability while leaving the foundational question unstated: what, in concrete terms, must still hold for the process to count as correct?

In practical system discussions, the same vocabulary is often used for quite different obligations. Replay, retention, immediate handling, idempotence, eventual consistency, and compensation all matter, but they do not matter in the same way for every flow. Work on event-management challenges and dataflow trade-offs suggests that correctness concerns are often more varied than the language around them admits [1, 3, 11]. Empirical work on event-based microservices points to the same difficulty in practice: events are used to carry application work across service boundaries, yet developers still struggle to design, inspect, and reason about those flows in ways that make the resulting process clear [1]. This thesis takes that difficulty as its starting point.

## 2.1 Event-driven work as process-level obligation

The unit of analysis here is not the event in isolation. It is the case or process whose acceptability may depend on what the event causes, preserves, updates, or triggers. An event is important because it participates in some larger piece of work. It may report a change in state, trigger a downstream action, carry work forward, or make recovery possible after a handling gap. The same technical treatment of an event can therefore be adequate in one flow and inadequate in another.

This is why event delivery alone is too narrow a view of reliability. A delivered event may arrive too late to help. A retained event may be replayable after the point at which its result can still count as correct. A missed state update may be harmless if later state legitimately overtakes it. A downstream action may still be wrong if it happens twice, even when every individual message was processed according to the mechanism's local rules. In each case, the relevant question is whether the process still satisfies the condition or conditions that make it acceptable.

This process-level view extends an end-to-end style of reasoning into event-driven process design. Saltzer, Reed, and Clark show that a lower-level mechanism may be useful and still be insufficient if the final requirement must be checked at the application boundary [4]. The same discipline applies here. Replay, retention, idempotence, and deferred settlement are not strong or weak in themselves. They are adequate or inadequate in relation to what the process must protect.

## 2.2 Mechanisms and the conditions they serve

The related work reviewed above makes one point especially important for this thesis: mechanisms do not explain their own reliability value. Publish-subscribe delivery, stream processing, recovery, idempotence, compensation, and transactional publication all provide useful capabilities, but none has a fixed process-level meaning on its own. Replay, for example, only says that emitted events can be processed again. Whether it protects correctness depends on the protected condition and on the handling model in which replay operates.

The same mechanism can therefore have different significance in different event-driven processes. Replay can recover emitted work after a downstream handling gap, but it cannot recover an event that was never emitted. Retention can preserve work for later handling, but preserved work may no longer matter after a deadline has passed. Deferred settlement can reduce harmful immediate duplication, but it also moves cost into later reconciliation. Producer-side safeguards matter when the event must enter the flow before any downstream mechanism can act on it.

A process-handling model is therefore not a single mechanism. It is the arrangement through which a process attempts to keep the relevant protected condition or connected set of protected conditions true. In the empirical part of the thesis, this idea is represented by three transport-anchored process-handling models: transient/immediate, retained/immediate, and retained/deferred. They do not exhaust event-driven handling. They make a controlled base contrast visible: a handling choice has to be judged by what the process must protect.

## 2.3 Three kinds of protected conditions in context

The three kinds of protected conditions used in the thesis are working distinctions rather than a complete taxonomy of process correctness. They bring into view obligations that prior work treats in different ways: timing and latency, acceptable state under asynchronous change, and safe downstream effects under retry, compensation, or transactional publication. The older safety/liveness distinction is useful background because it shows that correctness properties may involve different kinds of obligation, including what must not go wrong and what must eventually occur [12]. Read at the level of event-driven processes, these obligations turn timeliness, state consistency, recovery, idempotence, compensation, and transactional event publication into design pressures rather than isolated mechanisms [3, 5, 6, 11, 13].

These distinctions focus the analysis rather than label whole flows. They identify the condition, or connected set of conditions, that makes a process outcome acceptable. A real process may combine them: a payment authorisation may need to complete while the checkout remains valid and also avoid harmful repeated execution. The design task is therefore to state what would make the process outcome acceptable before judging whether a handling choice protects that requirement.

### 2.3.1 Timing and validity windows

Timing is already central to dataflow and service-latency work: eventual processing is not enough if the result arrives outside the window in which it can still matter [3, 9]. The relevant point is not that every event-driven process has a strict deadline, but that some outcomes lose their value once the application context has moved on. Delay matters when it threatens the condition that makes the outcome still useful. For deadline-constrained work, preserved work matters only where recovery can still arrive within the validity window. Replay or deferred settlement is valuable only if it keeps open a path to a timely acceptable outcome.

### 2.3.2 State divergence and acceptable state

Replication and state-consistency work make temporary divergence a familiar problem, but they also show why the significance of a missed update depends on the state obligation at stake [5]. Final convergence is not always enough to answer that question, because customers, operators, or later processes may have acted on the state exposed during the disturbance. Conversely, a missed intermediate update may matter little once later updates restore the state the process is meant to preserve or expose. The state-oriented question is therefore whether missed or delayed updates leave the visible or resulting state unacceptable, rather than whether every intermediate update was processed.

### 2.3.3 Downstream effects and duplicate control

The literature on sagas, idempotence, and transactional event publication makes downstream effects visible as obligations that can fail in different ways: later corrective work may be needed, retry may repeat the effect, or a committed source

change may fail to enter the event flow [6, 11, 13]. For required-effect work, those concerns meet in one process-level question: whether one downstream action happens reliably and without harmful duplication. If a valid event has been emitted, replay can recover a downstream handling gap. If the event was never emitted, downstream replay has no object to act on. Producer-side safeguards are therefore important, but in the empirical comparison they protect entry into the event flow rather than forming a fourth base model. In such flows, duplicate control is ordinary application-side discipline, with idempotent handling as the usual protection. Deferred settlement addresses a different question: it can reduce immediate duplicate side-effects by holding work for later reconciliation, at the cost of settlement delay and reconciliation burden.

## 2.4 Summary and relation to this thesis

The chapter’s central lesson for the thesis is that mechanism-level capability and process-level acceptability are related but not the same. The works discussed above show why event-driven reliability cannot be reduced to message movement alone. Publish-subscribe systems, recovery mechanisms, stream processing, compensation, coordination avoidance, and transactional event-handling patterns all offer ways to move, preserve, replay, coordinate, or settle work. For the argument developed here, the important point is that preserved work is not self-explanatory. A retained event, a replayed handler, a compensating step, or a transactional publication pattern has to be read against what the process still needs to achieve.

End-to-end reasoning is especially important to that reading. Saltzer, Reed, and Clark show that a mechanism may be useful at one level and still be insufficient unless the final requirement is protected where it can actually be checked [4]. In the same spirit, event-driven mechanisms may be necessary while still leaving process acceptability unsettled. A handling model is adequate only in relation to what the process must protect.

The same idea connects the thesis to work on coordination avoidance, CALM, and highly available transactions. Those works relate coordination and availability to the property or guarantee at stake, such as application-level invariants, monotonicity, or semantic transactional guarantees [14, 15, 16]. Stream-processing, dataflow, and online event-processing work add a related concern: correctness, latency, delivery, processing, and transactional guarantees have to be understood in relation to what they actually cover, especially when timing, later correction, external state, or side-effecting systems are involved [3, 10, 17].

The thesis takes up that relation by making the process-level requirement explicit before mechanism selection. In that reading, the focus moves from what the system can preserve, retry, or repair to what such handling can still make possible for the process. This gives the thesis its grounding in related work and its point of departure from it: reliability is judged by the requirement that makes the process outcome acceptable, and mechanisms by the part they play in meeting that requirement.

### 3.1 Research design

#### 3.1.1 Design-science framing

The thesis follows a design science research approach: it develops a boundary-first framework and examines it through a controlled empirical suite [18, 19]. The approach is useful because it separates the designed contribution from the evidence used to examine it. The framework is the central artifact: a process-level way of relating correctness boundaries, protected conditions, failure modes, process-handling models, and trade-offs. The guide developed in Chapter 5 gives this reasoning an applicable form. The controlled suite then exercises the framework under selected disturbances, so that the consequences of different handling choices can be observed against defined protected conditions.

This positioning matters because it keeps the design-science claim precise. The guide is not treated as a separately validated design method, and the suite is not treated as a platform benchmark. Instead, the method is organised around a more exact question: what becomes clearer when handling choices are judged against the protected condition or connected set of protected conditions that defines process acceptability? Table 3.1 states this methodological basis before the experimental details begin. It makes clear what is being designed, how it is exercised, and what kind of evidence the controlled suite can provide.

Table 3.1: Design-science role of the framework and empirical suite.

| Design-science element | Role in this thesis                                                                                                                                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Design problem         | Event-driven reliability is difficult to judge when the protected condition is implicit. The design problem is to make process acceptability explicit before judging the handling model.                                                  |
| Artifact               | The artifact is the boundary-first framework: a process-level way of relating correctness boundaries, protected conditions, failure modes, handling choices, and trade-offs. The Chapter 5 guide gives this reasoning an applicable form. |
| Demonstration          | The framework is exercised through a controlled empirical suite covering deadline-constrained, state-oriented, and required-effect processes under selected disturbances.                                                                 |
| Evaluation logic       | The suite is read through boundary-aligned evidence: whether work completes while it still matters, whether state becomes acceptable, whether the required effect occurs safely, and where recovery or settlement burden is moved.        |
| Claim boundary         | The evidence supports disciplined comparison within the controlled setting. It does not validate a universal design method, benchmark the underlying platform, or predict production behaviour in general.                                |

### 3.1.2 Controlled comparison and disturbance

The empirical suite is deliberately small. Its purpose is to make selected handling differences visible under controlled conditions, so that the comparison turns on the protected condition, the handling model, and the disturbance profile. Three choices define that instrument. First, the condition modules correspond to the protected conditions developed in the thesis: timely completion, acceptable state, and one required downstream effect. Second, the process-handling models provide a compact, transport-anchored contrast between transient/immediate, retained/immediate, and retained/deferred. Third, disturbances are introduced where delay, omission, duplicate pressure, or accumulated work can affect whether the relevant condition remains satisfiable.

Disturbance therefore has both a substantive and an analytical role. Substantively, it is where reliability concerns become consequential: delay, loss, duplicate pressure, omission, and backlog can change whether the process still reaches an acceptable outcome. Analytically, it makes differences between handling models visible. In an uninterrupted run, several handling models may appear acceptable. Under disturbance, the suite can show which work remains recoverable, which delays still matter, which failures remain outside downstream recovery, and which costs are moved into later settlement.

## 3.2 Implementation environment and platform choices

### 3.2.1 Runtime arrangement

The empirical suite is implemented as a small Docker Compose-managed event-processing environment. Docker Compose is suitable for this purpose because it

keeps the runtime arrangement explicit and repeatable: the producer, Redis service, consumers, settlement process, and reporting scripts can be isolated as separate services while still being run as one controlled suite [20]. This isolation helps keep the comparison focused on the configured handling model and disturbance profile rather than on incidental changes in how the experiment is launched.

### 3.2.2 Redis transport choices

Redis provides the event transport and retention layer used in the suite. Redis Pub/Sub is used as the suite’s transient event bus because it delivers messages to active subscribers without retaining them for later replay [21]. This matches the thesis’s transient/immediate handling model, where missed delivery cannot be recovered downstream. Redis Streams is suitable for the retained cases because emitted entries remain available after initial delivery and can be read again [22]. This supports the suite’s contrast between transient delivery, retained replay, and retained work used for deferred settlement within the same lightweight platform.

### 3.2.3 Services, reporting, and versions

The producer and consumer services implement the process logic, disturbance behaviour, and handling models examined in the suite. Producers emit the configured workload. Consumers either attempt work immediately as events arrive or are replayed, or preserve emitted work for a later settlement step. Reporting scripts collect run outputs and derive the endpoint, runtime, replay, state, and settlement measures used in the analysis. The reported figures were generated from these outputs with Matplotlib.

The reported runs were executed from the frozen project artifact in Docker Compose-managed containers using Docker Compose v5.0.2. Redis was provided as Redis v7.4.8 through the `redis:7-alpine` container image. The producer and consumer services were built from the `python:3.11-slim` container image and used the Python Redis client package `redis` at v5.2.1. These details support reproduction and inspection of the controlled suite. The point here bears stressing: Docker, Redis, and Python are parts of the experimental arrangement, not the objects of comparison.

## 3.3 Scenario and run structure

### 3.3.1 Condition modules and disturbance profiles

The suite is organised around the three kinds of protected conditions introduced in Chapter 1 and situated in relation to prior work in Chapter 2. Each module keeps the scenario structure small enough to interpret, while varying the disturbance most relevant to the condition being examined.

The deadline-constrained module examines timely completion. It uses four scenario families: baseline, moderate degradation, high degradation, and backlog shock.

Baseline provides the reference condition. Moderate and high degradation represent sustained consumer-side pressure at two intensities. Backlog shock represents a sharper disturbance in which work accumulates and must later be repaid. These scenarios test whether preserved or replayed work can still complete before the validity window closes.

The state-oriented module examines whether missed or delayed updates still matter to the resulting or visible state. The comparison is smaller than in the deadline module because the empirical question is narrower: whether replay contributes to final or exposed state adequacy when updates are missed or delayed, and whether later updates can overtake earlier misses.

The required-effect module examines whether one downstream action happens reliably and without harmful duplication. It uses handling gap, source omission, and duplicate pressure to separate three concerns: replayable downstream loss, producer-side non-emission, and repeated triggers bearing on the same downstream obligation. This separation is important because replay can recover emitted work after a downstream gap, but it has nothing to recover when the source never emits the event.

The disturbance profiles are therefore not a general taxonomy of failures. They are selected probes. Processing may slow down, work may accumulate, emitted work may be missed downstream, repeated triggers may bear on the same downstream obligation, or some intended cases may never enter the event flow. Each disturbance is included because it can change the relation between the handling model and the protected condition.

### 3.3.2 Scenario-cell construction

Each scenario cell combines a condition module, a disturbance profile, and a process-handling model. A single run then executes that configured cell from start to finish: the producer emits the workload, the disturbance affects handling according to the profile, the selected model processes or preserves the work, and the reporting scripts record both endpoint outcomes and runtime traces. This structure keeps the unit of comparison stable across the suite. The comparison examines repeated executions of the same event-driven process and protected condition under the same disturbance and handling model. The full scenario and parameter specification is reported in Appendix B, Table B.2.

The suite size follows from those contrasts rather than from a target count. The required-effect module is larger because the same downstream obligation can fail at different points: the event may be emitted and missed downstream, never emitted at all, or triggered repeatedly. The deadline-constrained module varies the intensity and shape of time pressure. The state-oriented module is smaller because its comparison is narrower: whether missed state remains relevant and whether replay changes the resulting state. Crossing these selected disturbances with the relevant handling models gives the 34 scenario cells summarised in Table 3.2.

### 3.3.3 Repetitions and stability

Each reported scenario cell is repeated ten times. The repetitions are used to avoid treating a single execution trace as decisive and to check that the comparative pattern

remains visible across runs. They are not used to support statistical generalisation beyond the controlled suite. Repeated full-suite executions produced stable aggregate figures and principal comparative patterns, with minor variation mainly in timing-sensitive runtime traces. Endpoint figures therefore use aggregate summaries where appropriate, while runtime figures show median traces when the shape of recovery or repayment over time is the point of interpretation. Appendix B gives the fuller account of suite run stability.

Table 3.2: Frozen empirical suite manifest.

| Module               | Scenario cells | Repetitions per cell | Completed runs | Role in the argument                                                                            |
|----------------------|----------------|----------------------|----------------|-------------------------------------------------------------------------------------------------|
| Deadline-constrained | 12             | 10                   | 120            | Examines repayment, expiry burden, and runtime-versus-endpoint contrast.                        |
| Required-effect      | 18             | 10                   | 180            | Examines replayable handling gaps, source omission, duplicate containment, and settlement cost. |
| State-oriented       | 4              | 10                   | 40             | Examines latest-state adequacy, outage-time exposure, and forward resumption.                   |
| <b>Total</b>         | <b>34</b>      | <b>10</b>            | <b>340</b>     | Frozen reporting suite used for the thesis comparison.                                          |

## 3.4 Operationalising the protected conditions

Table 3.3 shows how the three kinds of protected conditions are made observable in the suite. Each module isolates one defining empirical question, so that the handling models can be compared against the condition at stake.

Table 3.3: Operationalisation of protected conditions.

| Protected condition  | Empirical question                                                                       | Main observations                                                                                                                                     |
|----------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deadline-constrained | Can affected cases still reach a valid outcome before the window closes?                 | Attained and expired outcomes, unresolved and doomed burden, peak and integrated burden, repayability, and runtime traces.                            |
| State-oriented       | Does the resulting or visible state remain acceptable despite missed or delayed updates? | Final state adequacy, outage-time exposure, and later state overtaking missed updates.                                                                |
| Required-effect      | Does one downstream action happen reliably and without harmful duplication?              | Unattained effects, handling-gap recovery, source omission, duplicate side-effects, correction rewrites, reconciliation passes, and settlement delay. |

The measures are therefore tied to the protected condition rather than to generic signs of success. Deadline-constrained scenarios pair endpoint outcomes with runtime burden. State-oriented scenarios separate final adequacy from outage-time exposure. Required-effect scenarios separate downstream recovery, source omission, duplicate containment, and settlement cost.

### 3.5 Metrics and interpretation

The metrics are used to read endpoint outcomes together with the path by which those outcomes were reached. Endpoint measures show whether the relevant condition was met. Runtime and settlement measures show how that result came about: whether work accumulated, whether recovery arrived in time, whether state was visible during disturbance, and whether deferred settlement shifted cost into later reconciliation. Table 3.4 summarises the metric groups used in the analysis. The full operational definitions are provided in Appendix B, Table B.1.

Table 3.4: Metric groups by protected condition.

| Protected condition          | Main empirical reading                                                          | Supporting measures                                                                                                                         |
|------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Deadline-constrained         | Whether cases can still reach a valid outcome before the deadline closes.       | Deadline miss rate, completion-in-time rate, unresolved burden, doomed burden, repayment pattern, and resolution pulse.                     |
| State-oriented               | Whether final state adequacy and outage-time exposure tell the same story.      | Latest-state attainment rate and outage-exposed seen fraction.                                                                              |
| Required-effect              | Whether the downstream action happens reliably and without harmful duplication. | Required-effect attainment, received-event count, duplicate side-effects, correction rewrites, reconciliation passes, and settlement delay. |
| Cross-cutting interpretation | Whether the final outcome alone hides the path by which it was reached.         | Endpoint outcomes read together with runtime traces, replay signals, and settlement-cost measures.                                          |

The main interpretive point is that final outcomes are not enough on their own. Completion, convergence, or duplicate containment may look favourable while still arriving too late, exposing misleading state during a disturbance, or moving cost into later settlement. For that reason, the results are read through endpoint outcomes, runtime traces, and settlement measures together. This emphasis on runtime meaning is consistent with recovery-oriented thinking [8], tail-latency analysis [9], and recent benchmarking work on fault-recovery behaviour [23, 24].

### 3.6 Reporting discipline and scope

The reported results come from the frozen suite described above: fixed scenario definitions, fixed containerised roles, fixed output extraction, fixed plotting conventions, and fixed reporting measures. The project artifact preserves the run specifications, raw run outputs, derived metric summaries, aggregate outputs, and plotting inputs. This supports traceability from the reported figures back to the controlled suite. The suite is modest in scale and in the load it places on the host machine. Host-machine context is reported in Appendix B for reproducibility, but the host specifications are not treated as variables in the comparison. The analysis relies on the comparative patterns produced by the suite, while allowing that exact timing traces may vary slightly across runs and hosts.

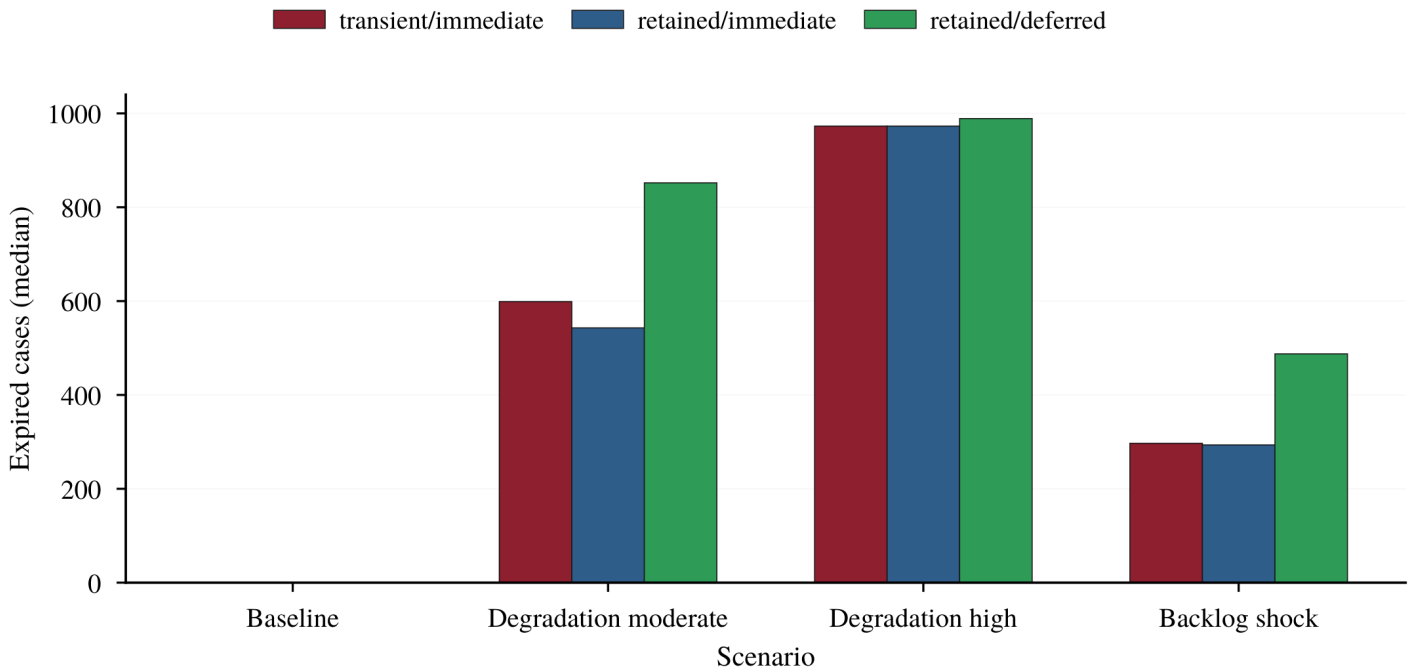
Preliminary runs were used to check that the main comparative patterns remained stable enough to support the argument. The thesis therefore treats the empirical material as a controlled comparison across defined conditions. The measured values are read as evidence about the relative behaviour of the handling models under the

defined disturbances, not as transferable performance values for external systems or workloads. The narrower claim is that the value of replay, retention, immediate handling, and deferred settlement depends on the protected condition at stake.

The results are organised by protected condition. Each section asks how the handling models behave when the relevant condition is deadline-constrained, state-oriented, or required-effect. Across the chapter, outcomes are interpreted through the condition at stake, including whether recovery still arrived in time, whether recovered state still mattered, and whether duplicate containment left an acceptable settlement burden.

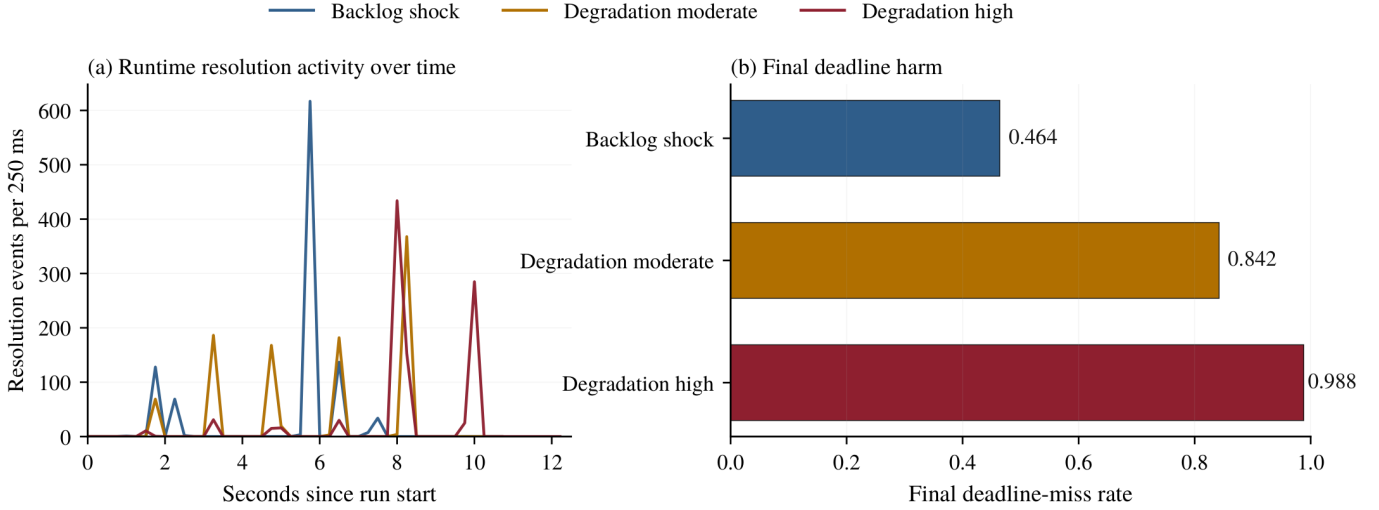
## 4.1 Deadline-constrained results

The central question in this section is whether cases affected by disturbance can still be brought to a valid outcome before the deadline passes. From that standpoint, backlog becomes analytically important when it threatens timely completion. The key issue is not simply whether work remains available, but whether unsettled work can still be repaid before the validity window closes [3, 9].



**Figure 4.1:** Deadline outcome burden by deadline scenario. Bars show median expired cases across ten repetitions for baseline, backlog shock, moderate degradation, and high degradation under the three process-handling models.

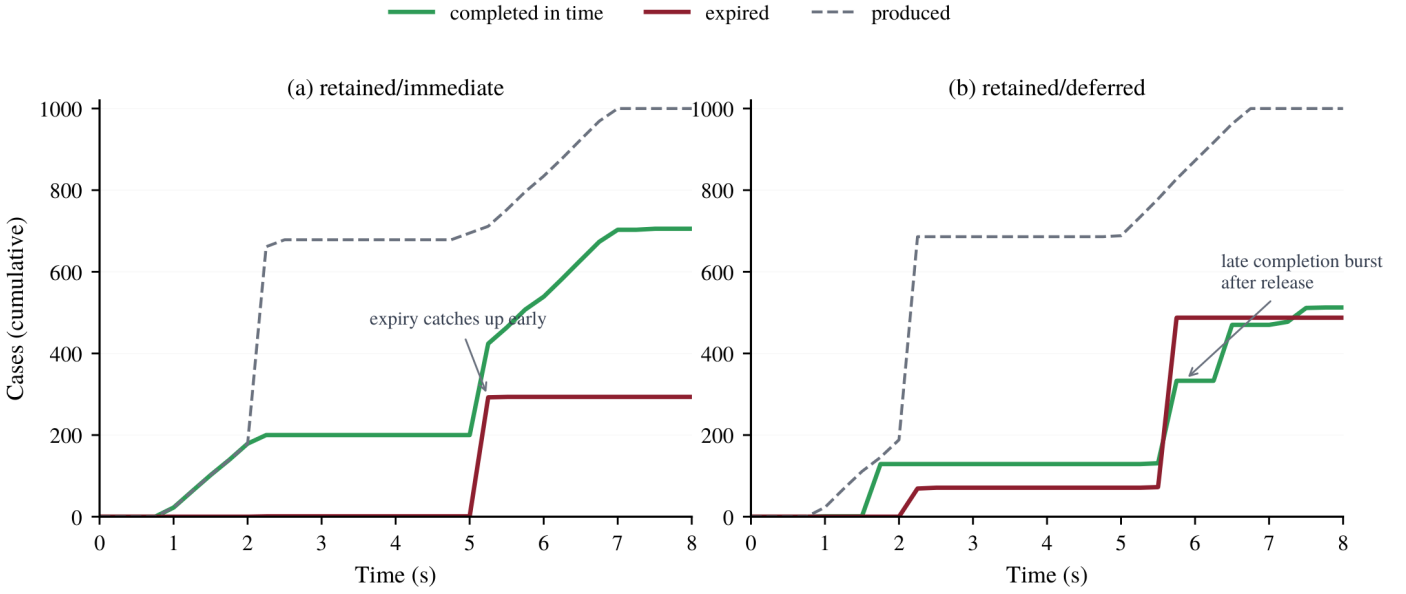
Figure 4.1 shows that deadline harm is shaped by repayability, not by disturbance intensity alone. Baseline produces no deadline burden, while backlog shock and sustained degradation produce different levels of expiry. A short, sharp disruption can be less damaging than a sustained slowdown if enough time remains for repayment. For deadline-constrained work, preserved work matters only if it can still complete before the deadline closes.



**Figure 4.2:** Runtime resolution activity and final deadline burden under retained/deferred handling. Panel (a) shows completions and expiries per 250 ms bin as median traces across the frozen ten-repetition batch, while panel (b) shows the corresponding median final deadline-miss rate.

Figure 4.2 explains why. Under retained/deferred handling, backlog shock produces the most visible burst of resolution activity, but not the greatest final deadline harm. Moderate and high degradation are less visually dramatic, yet they leave more cases beyond the deadline because they reduce the time available for repayment throughout the run. The figure therefore shows why runtime activity and endpoint damage have to be interpreted together.

Within the configured disturbance profiles and expiry window, the value of the contrast is illustrative: it shows how peak runtime pressure and final deadline harm can diverge when accumulated work can still be repaid before expiry.



**Figure 4.3:** Cumulative accounting for the backlog-shock case. The curves show produced cases, in-time completions, and expiries as median cumulative traces across the frozen ten-repetition batch.

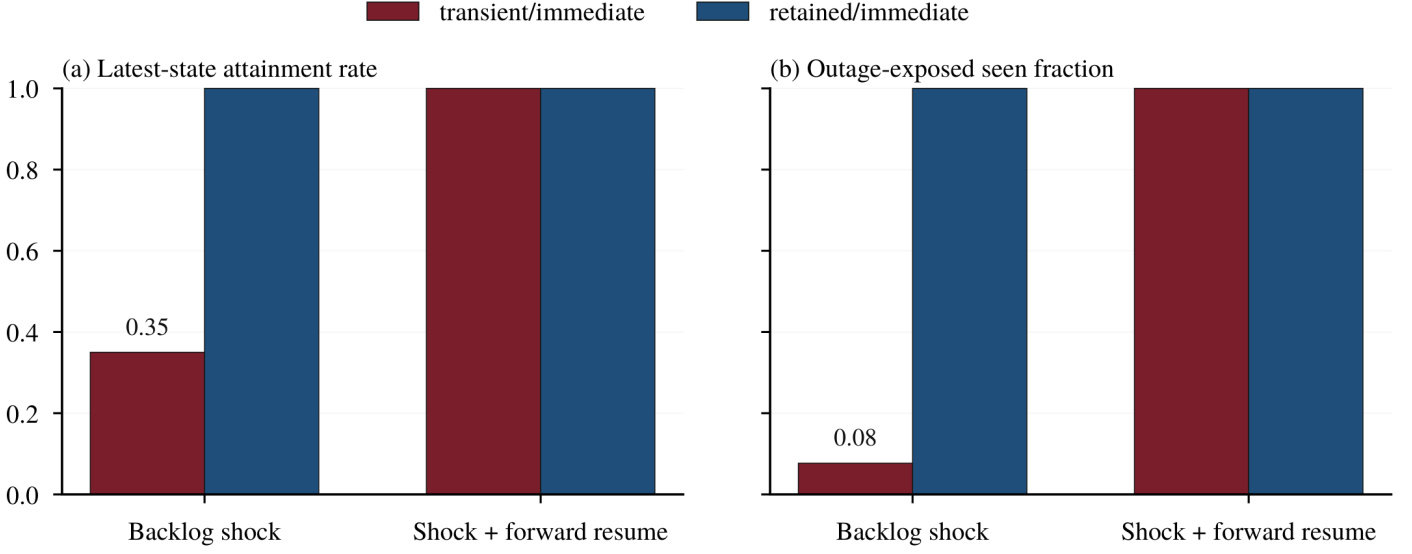
Figure 4.3 gives a more concrete accounting of the backlog-shock case. The retained/deferred trace includes a visible late completion burst after release, but much of the deadline damage has already occurred by then. Recovery activity is therefore not enough on its own. For this boundary, it matters only if it arrives while the outcome can still count as correct.

Taken together, the figures show that deadline-constrained work is governed by repayability before expiry, not by preserved work or peak runtime pressure alone. Retention and replay can keep work available, but the handling model should be judged by whether recovery can still arrive in time for correctness to hold.

## 4.2 State-oriented results

State-oriented work raises a different correctness question: whether the state represented by the process remains acceptable despite missed or delayed updates. From that standpoint, replay is not valuable in itself. Its value depends on whether a missed update remains relevant to the state the process is required to preserve [5, 25].

Two measures separate final adequacy from outage-time exposure. Latest-state attainment records whether the process reaches the expected latest version for the case. Outage-exposed seen fraction records whether updates during the disturbance were observed. The distinction matters because a process may end in an acceptable state even when some intermediate state was missed, while in other cases the missed state may remain relevant.



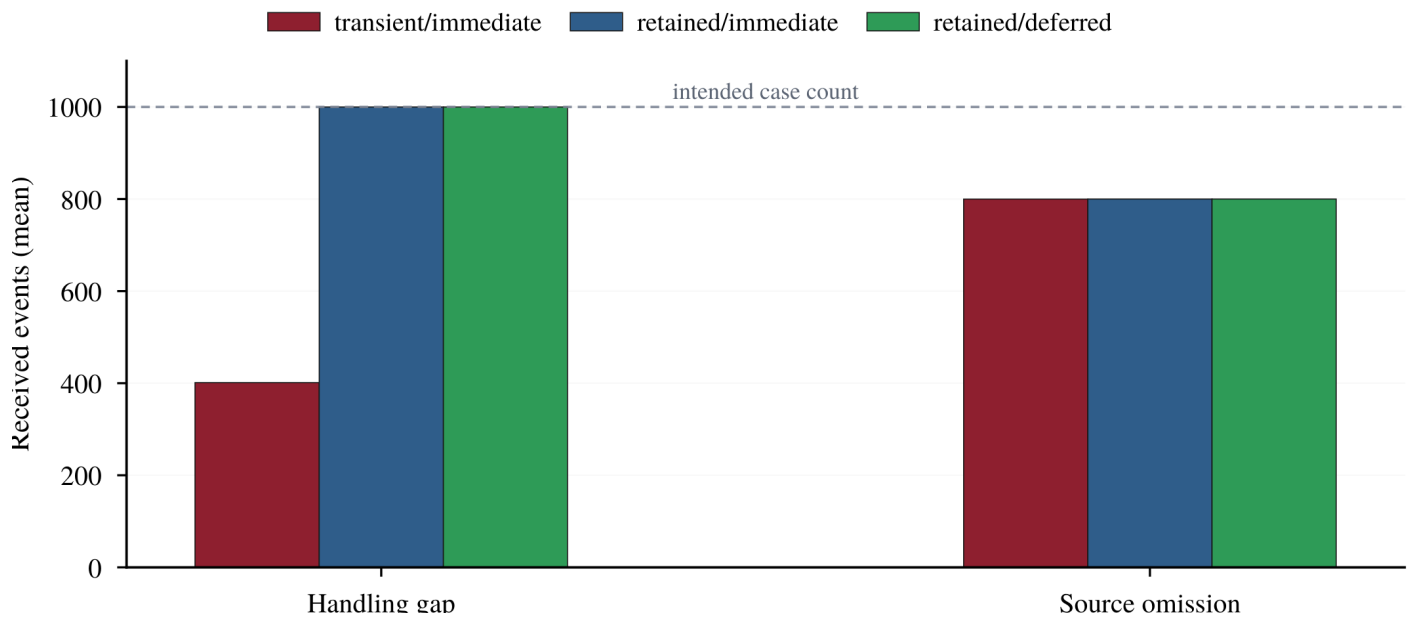
**Figure 4.4:** State-oriented adequacy under backlog shock and forward resume. The figure compares latest-state attainment and outage-exposed seen fraction for transient/immediate and retained/immediate handling.

Figure 4.4 shows why replay relevance is conditional on the state boundary. Under backlog shock, transient handling loses state-relevant updates, while retained handling preserves them for replay. Under forward resume, later updates overtake the missed information, and both models reach full latest-state attainment. The same missed-update pattern therefore has different meaning depending on the state the process is required to preserve. Replay matters when it restores state that still counts. It matters less when newer state has already made the missed update obsolete.

Taken together, the state-oriented results suggest a restrained design implication: replay is useful when it restores state that still matters. In the continuous-update setting examined here, later updates can overtake earlier misses, while infrequent updates or long-lived state gaps may still justify retained replay. The handling model should therefore be judged by whether replay contributes to the state condition, not by event deficit or replay completion alone.

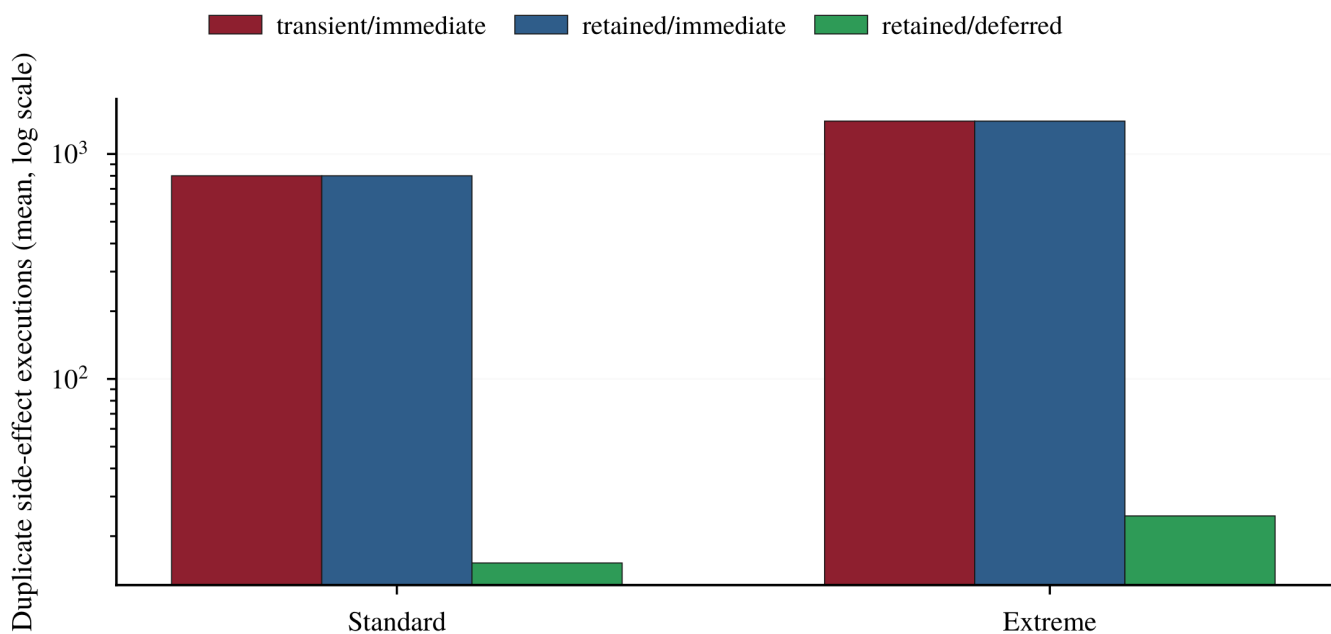
### 4.3 Required-effect results

Required-effect work asks whether one downstream action happens reliably and without harmful duplication. The first distinction is where the failure occurs. A downstream handling gap can remain recoverable if the event has entered the flow, while source omission leaves replay and retained handling with nothing to act on. Duplicate pressure raises a separate question: whether repeated triggers lead to harmful repeated effects, and what cost is introduced when the handling model contains them [6, 11].



**Figure 4.5:** Source omission boundary measured as received events. The figure compares handling gap and source omission across the three process-handling models, with the intended case count shown as a reference.

Figure 4.5 marks the boundary of downstream recovery. Its value is that it shows where replay has an object to act on and where it does not. In a handling gap, the event has entered the flow and retained handling can recover what transient handling misses. In source omission, some cases are never emitted as events, so the failure lies before replay can help. The figure therefore separates downstream recoverability from producer-side integrity, which is essential for required-effect work.



**Figure 4.6:** Duplicate-pressure containment. The figure shows duplicate downstream side-effects under duplicate pressure for immediate and deferred handling.

Figure 4.6 addresses another part of the required-effect boundary: immediate repeated execution versus deferred settlement. In the retained/deferred model used in the suite, repeated triggers are absorbed before final settlement, sharply reducing duplicate downstream execution. Table 4.1 reports the companion cost as rounded means across the frozen ten-repetition batch.

Table 4.1: Duplicate-containment trade-off under deferred settlement.

| Severity | Immediate<br>dup. effects | Deferred<br>dup. effects | Reconciliation<br>passes | Correction<br>rewrites | Settlement<br>delay (s) |
|----------|---------------------------|--------------------------|--------------------------|------------------------|-------------------------|
| Standard | 800                       | 15                       | 33                       | 785                    | 0.10                    |
| Extreme  | 1400                      | 25                       | 37                       | 1375                   | 0.06                    |

Duplicate side-effects fall substantially under both pressure levels, while reconciliation work, correction rewrites, and settlement delay increase. Deferred settlement therefore changes the burden rather than removing it, moving it out of immediate repeated execution and into later settlement work.

Taken together, the required-effect results separate concerns that are easy to blur. Replay is relevant only after a valid source event exists, while duplicate containment raises a separate settlement trade-off. In the duplicate-pressure scenario examined here, the retained/deferred model reduces repeated downstream effects by moving work into reconciliation, correction, and settlement delay. A handling model for required-effect work should therefore be judged by both the location of the omission risk and the cost of postponing settlement.

## 4.4 Remaining possibility after disturbance

Across the preceding results, a central question is brought into view: after disturbance, is there still a path to an acceptable outcome? Preserved work matters only where it keeps that path open. A retained, replayed, delayed, or deferred action has value only if it can still help make the process acceptable.

Disturbance gives the comparison its force because it exposes the distance between preserved work and remaining possibility. A retained event may be present, replayable, and technically intact, yet its value depends on what can still be made true with it. In some scenarios, the retained event keeps a path to recovery open. In others, it preserves work whose usefulness has expired, or whose relevance has already been overtaken.

Retention therefore receives a more exact interpretation. It preserves an object of recovery, while the protected condition determines what that object is worth. A replay path has value when it can still arrive in time, restore state that still matters, or re-drive an owed downstream effect after emission. Outside those conditions, the mechanism may continue to function while the process has moved beyond what that mechanism can repair. Preservation and usefulness can come apart.

The required-effect scenarios mark the hardest edge of that separation. Once an event has entered the flow, retained handling can make downstream gaps visible and recoverable. Before successful emission, however, downstream machinery is blind: there is no event to retain, replay, inspect, or settle. Source integrity and consumer-side recovery therefore protect different parts of the path by which a required effect

becomes possible. This distinction can easily be blurred in ordinary parlance about missing events, where source omission and downstream handling gaps can be conflated into the same kind of absence.

Deferred settlement shows a different movement. It can turn immediate duplicate harm into accountable work, but the burden has changed form rather than disappeared. It reappears as reconciliation, correction, and settlement delay. That conversion may be valuable where later correction is preferable to repeated downstream effects, but the results make the cost visible as part of the handling choice rather than as an external afterthought.

Across these cases, disturbance shows whether handling keeps the path to acceptability open, preserves work whose practical value has faded, or moves the remaining burden into later settlement.

The controlled empirical suite gives the discussion a way to examine the research questions under defined disturbances. It helps illuminate how different handling models shape recovery, acceptability, and settlement. The chapter first addresses what counts as correct in the examined kinds of event-driven work, then considers what the comparison shows about the handling models, and finally uses that comparison to distill practical design guidance about proportionate handling choices. That guidance is later made more concrete in the form of a boundary-first guide and accompanying design examples. The later sections state the limits of the claims and consider the professional significance of making process-handling choices explicit, while Chapter 6 returns to the broader research context.

### 5.1 Answer to RQ1: protected conditions and process acceptability

*RQ1. Which kinds of protected conditions can be used to reason about correctness in the event-driven processes examined here?*

The examined processes show three useful kinds of protected conditions through which process correctness can be judged: deadline-constrained, state-oriented, and required-effect.

For deadline-constrained work, correctness turns on whether affected cases can still reach a valid outcome before the window closes. Delay matters because it can change the status of the outcome itself. A case may eventually complete and still fail if completion arrives after the point at which the result no longer counts. Time is therefore part of correctness, not merely a performance concern.

For state-oriented work, correctness turns on whether missed or delayed updates still matter to the state the process is required to preserve or expose. A missed intermediate update is not necessarily a lasting failure, since later state may legitimately overtake it. Whether that matters depends on the specific state condition: some state obligations are satisfied by the latest valid state, while others are affected by stale or misleading state during the disturbance. The central question is whether the state condition still holds, not necessarily whether every update has been observed or replayed.

For required-effect work, correctness turns on whether one downstream action happens reliably and without harmful duplication. The first distinction is where the failure occurs. Once an event has entered the flow, replay may recover a downstream

handling gap. Before emission, there is no event for replay to recover. Repeated triggers create a separate risk: the action may happen more than once, so duplicate control and settlement cost become part of judging whether the condition is protected.

Taken together, these distinctions give correctness a process-level form. A process is correct when the condition or connected set of conditions that defines its boundary still holds: the outcome arrived in time, the relevant state remained acceptable, or the required downstream action happened without being lost or harmfully repeated. That is the foundation for judging the handling models discussed in the next section.

## 5.2 Answer to RQ2: handling models under controlled disturbance

*RQ2. How do the selected process-handling models shape preservation, replay, repayment, and settlement under controlled disturbance?*

The selected process-handling models shaped preservation, replay, repayment, and settlement by changing what could still be recovered, when recovery could still matter, and where remaining burden was moved. Three patterns stand out in the results.

For deadline-constrained work, peak pressure matters less than whether unsettled work can still be repaid before expiry. The retained/deferred backlog-shock case makes this visible: the most dramatic runtime burst does not produce the greatest final deadline harm, because endpoint damage depends on whether accumulated work can still complete in time.

For state-oriented work, replay value depends on whether a missed update still matters to the state condition being protected. In the continuous-update setting examined here, later updates can overtake earlier missed state, making transient handling sufficient. In other state-oriented settings, where missed updates remain relevant for longer, retained replay may still be the more appropriate model.

For required-effect work, downstream handling gaps and source omission are not the same kind of failure. Once an event has entered the flow, retained replay can recover missed downstream handling. If the event was never emitted, replay has no object to act on. Deferred settlement adds a different trade-off: it can reduce immediate downstream pressure, but it does so by moving part of the burden into later settlement.

These modest distinctions sharpen the question of handling choice. They show that the same handling model can preserve what matters in one kind of work, arrive too late in another, or address the wrong failure altogether.

A further lesson is that recovery signs have to be read through the condition they are supposed to satisfy. A cleared backlog, a replayed event, a converged final state, or a lower duplicate count may all look favourable. None is decisive on its own. The relevant question is narrower: did the work still arrive in time, did the recovered state still matter, and did duplicate containment leave an acceptable settlement burden?

This is why the empirical suite follows both endpoint outcomes and the route by which those outcomes were reached. Endpoint results show where the process

ended. Runtime and settlement traces show whether that endpoint still satisfied the condition at stake, or whether correctness had already been lost, superseded, or shifted into later work. The distinction is small in wording but important in design: a system can appear to recover while still failing the condition the process was meant to protect.

## 5.3 Answer to RQ3: correctness boundaries and handling choice

*RQ3. How can explicit correctness boundaries guide process-handling choices in event-driven design?*

Explicit correctness boundaries guide process-handling choices by making each handling model answerable to what makes the process outcome acceptable. They require the process under deliberation to be scoped so that its protected condition, or connected set of protected conditions, can be stated coherently. Replay, retention, settlement, and supporting safeguards can then be judged by whether they fit what must hold for that process. The guide below gives this reasoning a practical form.

### 5.3.1 Boundary-first design guide

The guide is applied to one event-driven process at a time. A larger application may contain several such processes, and a single process may have a boundary defined by one protected condition or by a connected set of protected conditions. This keeps the judgment close to the handling choice itself. First state what would make the process acceptable. Then ask how delay, loss, stale state, duplicate action, settlement delay, or source omission could make the process unacceptable. Only then should the handling model and any supporting safeguards be judged as proportionate.

Table 5.1 presents the guide in a compact design-facing form. It keeps the main deliberative move visible: begin with the boundary question, read the design implication, and then judge which handling consideration follows.

Table 5.1: Boundary questions and handling considerations.

| Boundary question                                                   | Design implication                                                                                                                                        | Suitable handling consideration                                                                                                                                                |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Can the connected conditions all hold within this process boundary? | Connected conditions are joint requirements. If they cannot hold together, the issue is boundary coherence rather than only the choice of handling model. | Rescope the process or split the larger flow into connected processes before judging replay, retention, settlement, or supporting safeguards.                                  |
| Must the process complete before a deadline?                        | Recovery is useful only while it can still arrive before expiry.                                                                                          | Retained handling can help, but only with deadline-aware checks. Deferred settlement is suitable only where later completion still counts.                                     |
| Does missed state still matter?                                     | Replay matters when a missed update would leave the visible or resulting state unacceptable.                                                              | Retained/immediate handling helps where stale state remains harmful. Direct reads or bounded refresh may be enough where the representation can stay close to canonical state. |
| Must one downstream effect happen safely once?                      | Recovery must address downstream handling gaps without creating harmful repetition.                                                                       | Retained handling may recover emitted work. Idempotence or deferred settlement may be needed where retry could duplicate the effect.                                           |
| Could the source fail to emit the event?                            | Downstream replay cannot recover work that never entered the flow.                                                                                        | Treat this as a producer-integrity concern, such as transactional publication or another source-side safeguard.                                                                |
| Is deferred settlement acceptable?                                  | Duplicate pressure may be reduced, but the burden moves into later reconciliation and settlement delay.                                                   | Retained/deferred handling is suitable only where that delay and settlement burden are acceptable.                                                                             |

Appendix A, Table A.1, provides the fuller guide. It expands the same reasoning into the checks used throughout the thesis, including deadline sensitivity, state relevance, downstream recovery, duplicate harm, settlement tolerance, and source omission. Supporting safeguards are treated there as controls that may be needed for the stated condition or connected set of conditions to be protected.

Source omission is included as a check because it lies before downstream handling. It is treated separately below to keep producer-side integrity distinct from the handling models compared in the empirical suite.

### 5.3.2 Producer omission and producer-integrity controls

Downstream recovery depends on the event having entered the flow. Once an event has entered the flow, retained replay can recover downstream handling loss. If the event was never emitted, replay has no object to act on. Where omission would materially affect the outcome, producer-side integrity controls should be considered before downstream recovery is treated as sufficient.

A transactional outbox is one example. It ties event publication to the committed change that gives rise to it, helping ensure that an event is not lost before it enters the flow [13]. From that point onward, the framework becomes applicable to questions of preservation, replay, delay, duplication, and settlement.

### 5.3.3 The cancellation case as a process chain

The controlled scenarios separated the protected conditions so that each kind of reasoning could be seen on its own. That separation provides analytical and pedagogical value, but real application design is usually less tidy. In a fuller setting, an accepted cancellation may first have to be established as a source fact, and that fact may connect several protected conditions in the work that follows: updating availability, offering the place to someone on a waiting list, or issuing a credit. The same customer action can therefore give rise to a small chain of related event-driven processes, not a single uniform reliability problem. At this point, it is useful to return to the fitness-center booking example introduced in Section 1.1.1 and read it as a small chain of related processes.

#### 5.3.3.1 Establishing the source fact

When a customer cancels a class booking, the chain begins by establishing the cancellation itself. The system has to decide whether the request becomes a recorded cancellation. The result should be clear and prompt: rejected, recorded, or visibly failed. If the cancellation is accepted, the application records a cancellation fact in its primary state. That fact becomes the source from which later availability, waiting-list, and credit processes can proceed.

#### 5.3.3.2 Following the downstream processes

Once that fact exists, later processes begin from it. The capacity view or calendar should show the released place. The waiting-list process should offer the place while it can still be taken. The credit process should issue any qualifying credit. The confirmation should tell the customer what happened. In ordinary product language, these can all be described as part of the cancellation flow.

That product-level description is useful because it preserves the user's view of the cancellation. Design reasoning, however, has to translate it into process-level questions. The cancellation commit, the visible representation, the waiting-list offer, the credit, and the confirmation are connected, but each carries a different condition of acceptability. Boundary-first reasoning keeps the chain together while asking what must still hold at each point, and what kind of handling is needed to protect it.

#### 5.3.3.3 Judging each process by what must hold

Read in that order, each event-driven process can be judged by the protected condition at stake and by the handling choice that protects it. The cancellation-commit process has to make an accepted cancellation durable and unambiguous, with enough timeliness for later handling to proceed from a settled source fact, so its protection belongs close to the decisive database update. The capacity or calendar representation is state-oriented: what matters is acceptable closeness to the canonical booking state, which may be better served by direct reads or bounded refresh than by replaying every missed update. The waiting-list offer is deadline-constrained, so retained handling and retry matter only while the released place can still be taken. The credit process is required-effect work, so recovery must ensure that an owed credit is issued

without being lost or repeated. If the service promises confirmation, sending it becomes required-effect work with a temporal dimension: the message should arrive promptly enough to reassure the customer and guide any next action. The system should also make unresolved downstream work visible, since the cancellation may already be true while some consequence of it remains incomplete.

The gain is modest and specific. The framework supplies an order of deliberation: scope each process, state what must still hold, and judge the handling choice against the condition or conditions at stake. In a fuller application setting, this can keep the canonical source process, the downstream processes, and their handling choices intelligible as a connected chain.

### 5.3.4 Connected conditions and boundary coherence

Connected protected conditions are joint requirements rather than competing labels. When several conditions define a process boundary, they must be capable of holding together for the process outcome to count as acceptable. If they cannot hold together, stronger handling does not solve the problem. The process under deliberation may instead need to be reconsidered, including whether some condition belongs to another connected process in the larger flow.

This is the purpose of the guide's boundary-coherence check: before choosing a handling model, the designer should ask whether the connected conditions can hold within the process being judged. The check does not add another kind of protected condition: it asks whether the conditions already named have been attached to a process in which they can coherently apply.

#### 5.3.4.1 Source facts and downstream scope

The cancellation-chain example can also help illustrate this point. A cancellation request can start the upstream work of deciding whether the cancellation should be accepted. It does not by itself start a valid credit-issuance process. For credit issuance, the relevant source fact is narrower: the cancellation has been accepted and found credit-eligible. Once that fact exists, the process can be judged by its own correctness boundary: the credit should be issued once, to the right customer, without harmful duplication, and within whatever usefulness window applies.

#### 5.3.4.2 Recognising a boundary mismatch

A faulty formulation would treat the cancellation request itself as the start of credit issuance while also requiring that no credit be issued unless the cancellation has already been accepted and found credit-eligible. Both requirements are understandable on their own: credit should be issued promptly, and credit should not be issued without eligibility. The problem is that they have been made to govern the same process from the wrong starting point. The process is asked to behave as if credit issuance is already valid, while also being constrained by the fact that it is not yet valid.

That is a boundary-coherence problem, not inadequate handling within a sound process boundary. The boundary has joined requirements that belong to different

points in the larger flow. The system must first establish credit eligibility before credit issuance can be judged as a separate downstream process with its own correctness boundary.

Boundary-first reasoning here exposes the mismatch. The larger flow should first establish the accepted, credit-eligible cancellation. Credit issuance then comes into play as a separate downstream process to be judged. Its correctness boundary can be stated coherently: the credit should be issued once, without harmful duplication, and within whatever usefulness window applies.

### 5.3.5 Further examples

The same reading can be applied more compactly in other ordinary event-driven domains. The following examples are shorter than the fitness-center booking case. Their role is to show how the same process-level discipline transfers when related processes share an order or delivery context.

### 5.3.6 Food delivery service

In a food delivery service, a placed order may similarly set several later processes in motion. The restaurant may need to accept the order, a courier may need to be assigned, the customer may need a useful status view, and refund or payout work may follow. These processes are connected by the order, but they are not made reliable in the same way.

A reasonable boundary reading might treat the status shown to the customer as state-oriented: missed intermediate updates may matter little if the display soon returns to an adequate account of preparation, pickup, or delivery. Restaurant acceptance, courier assignment, and payment authorisation often have a deadline-constrained character, since delay can leave the restaurant without time to prepare, the courier without a useful pickup, or the customer without a confirmed order. Refunds and courier or restaurant payout are usually closer to required-effect work: once owed, they should be completed without being lost or repeated, and what matters is reliable eventual settlement rather than immediate completion.

The practical implication is that one order should not be treated as one reliability problem. It can give rise to several connected processes, and the handling choice should follow the work each process is meant to make acceptable.

### 5.3.7 E-commerce checkout and order processing

E-commerce checkout gives the same issue a slightly different shape. To the customer, checkout may appear as one order journey. Internally, however, the system may have to coordinate stock, payment authorisation, invoice generation, refunds, and shipping updates around the facts that have been committed.

Stock and order-status views are state-oriented: they should remain close enough to the real order or inventory state to support customer and operational decisions. Payment authorisation is deadline-constrained, because it has to complete while the checkout session, authorisation context, and customer intent still hold together. Invoice generation, refunds, and shipping updates are required-effect work once the

relevant order fact exists: they should not disappear downstream, and they should not be repeated harmfully.

The framework makes that mixed order journey easier to reason about: stock and status concern usable state, payment concerns timely action, and invoices, refunds, and shipping updates concern downstream work that must still be completed.

### 5.3.8 Scope and realism

The guide is meant for ordinary systems in which event-driven processes carry part of the work that matters to an outcome. The authoritative business record may remain in a transactional database, and many important decisions may still be made through ordinary application logic. What matters for the guide is narrower: whether a particular event-driven process affects whether a case reaches an acceptable result.

This defines the guide's scope. It examines one event-driven process at a time: what boundary the process must satisfy, how the conditions defining that boundary could fail, and which handling model is proportionate. Applied repeatedly, the same discipline can give a clearer view of the system's event processing as a whole, while the judgment remains process-level.

The guide is modest by design. It is most useful where ordinary design language compresses different obligations into one flow: timely action, reliable completion, recoverable state, tolerable delay, or harmful repetition. The framework gives those distinctions a more explicit shape without replacing the wider architectural judgment needed in a real system.

## 5.4 Limitations and claim boundaries

### 5.4.1 Framework scope

The claims made here are restrained by design. The framework relates process-handling models to explicit process-level correctness requirements. The three kinds of protected conditions used in the thesis are working distinctions, not a complete taxonomy of process correctness. They bring deadline validity, state relevance, and required downstream effects into view, while leaving room for other correctness concerns to require additional distinctions, or to cut across the ones used here.

### 5.4.2 Empirical scope

The empirical work has the same restrained scope. The suite uses Redis Pub/Sub and Redis Streams as controlled representatives of transient and retained event handling. Redis serves the comparison by making transient and retained handling observable under controlled conditions: Redis itself is not being assessed. It does not compare alternative broker or streaming platforms, such as Kafka, RabbitMQ, or managed cloud queues, and it is not presented as a platform benchmark. The workload is synthetic, the disturbance profiles are simplified, and the scenario parameters are chosen to make selected handling differences visible under fixed conditions. The results should therefore be read modestly. Their value lies in the patterns the controlled

scenarios make visible, rather than in the particular measurements taken from this suite.

The repeated runs should be read in the same way. Ten repetitions per scenario cell reduce dependence on a single execution trace and check whether the main comparative patterns remain visible. They strengthen confidence in the stability of the observed patterns within the suite, but they do not support broad statistical generalisation or claims about operational performance outside the controlled setting.

### 5.4.3 Guide and architectural scope

The design guide should also be read accordingly. It gives the framework a compact working form, but the study does not evaluate it as a practitioner method, compare it with alternative design procedures, or test it in real architecture reviews or incident investigations. Its role here is more modest but still central to the thesis: to show how a protected condition, a handling model, and the resulting trade-off can be brought into view together.

Nor does the thesis rank the handling models in general. Retention, replay, immediate handling, and deferred settlement take their value from the condition at stake. Deferred settlement, for example, is appropriate only where postponing settlement is preferable to allowing immediate duplicate or repeated downstream effects. The claim is narrower: explicit correctness reasoning gives a clearer basis for judging which handling model fits the process's protected condition or conditions.

Finally, the framework remains process-level guidance. Applied across the event-driven processes in a system, it can inform broader architectural judgment, but it does not replace the need to reason about data ownership, service boundaries, operational constraints, monitoring, recovery practice, ordering, or cross-process consistency. Its purpose is to discipline one part of that judgment: the relation between what a process must protect and how its event-driven handling should be arranged.

## 5.5 Ethical and professional considerations

The empirical work in this thesis does not involve human participants, personal data, or production systems, so its direct research-ethical risk is limited. The more relevant issue is professional. Reliability decisions in event-driven systems shape which failures are prevented, which are tolerated, and which are shifted into later handling. In real systems, those choices may affect people and organisations who did not choose the handling model but still bear the consequences of its limits.

This matters most where the burdens of failure are unevenly distributed. A delayed refund, missed notification, duplicated charge, or stale operational state may impose costs on customers, workers, or downstream parties who did not choose the handling model. The framework cannot decide how those costs should be balanced. It can, however, make clearer what makes the process outcome acceptable and how the handling choice assigns responsibility when that outcome is disturbed.

The framework's professional relevance is narrower and more practical. It makes reliability reasoning more explicit by connecting the acceptability of a process outcome to the handling choice that carries responsibility for it under disturbance. That can support responsible design by clarifying what guarantee was intended, what risk was accepted, and where recovery or settlement work was expected to occur.

The practical implication is that a handling model should not be justified only by technical convenience or available infrastructure support. It should be justified in relation to what the process must protect, and to the consequences if that protection fails. Making that reasoning explicit does not remove the need for wider ethical and architectural judgment. It does, however, make clearer what the handling choice is meant to secure and where the burden may fall if that protection fails.

## Chapter 6

---

# Conclusions and Future Work

## 6.1 Future Work

Future work can move the argument from controlled comparison toward the settings in which event-driven handling choices are actually made.

### 6.1.1 Design practice and guide use

One direction is to study design practice itself. Architecture discussions, design reviews, and incident reports could show how teams turn business or system requirements into process-level judgments about what must remain true, where recovery is expected to occur, and what settlement cost is acceptable. This would test whether the reasoning made explicit in the thesis reflects judgments that practitioners already make implicitly, and where the framework would need adjustment.

Another direction is to evaluate the design guide as a practical aid. Such a study could ask whether the guide helps teams state protected conditions, compare handling models, and recognise mixed obligations within the same application. The relevant question would not be whether the guide produces one correct answer, but whether it makes the grounds of a handling choice clearer and easier to discuss.

### 6.1.2 Platform mechanisms and operational judgment

Modern event-streaming and stream-processing platforms are a promising setting for this work because they move more reliability work into infrastructure. That shift can make a design look settled before its process-level obligation has been made explicit. A platform may keep work available, but availability is not yet acceptability: the process still has to be judged against the correctness boundary that makes its outcome acceptable. Future work could examine how teams carry that judgment across the boundary between platform configuration, application logic, and operational recovery.

Future work could also extend the framework into operations. Once a process-level correctness boundary has been stated, monitoring and incident review can ask whether that boundary was maintained during disturbance, not only whether events moved through the system. This would shift attention from activity alone to outcome evidence: what remained acceptable, what failed to hold, and what recovery or settlement work was still needed.

### 6.1.3 Connected conditions and framework extension

Further work could examine cases where connected conditions stretch across more than one process. The difficult question is where each condition becomes applicable. A downstream action may depend on an upstream fact that first has to be established, as in the cancellation chain. Similar multi-step flows would test whether the framework can keep boundary coherence visible when correctness is distributed across a larger chain rather than contained within one process alone.

Together, these directions would place the framework in real design, platform, and operational settings. They would show how explicit correctness boundaries clarify handling choices, and where the framework would need further refinement for broader use.

## 6.2 Findings and contribution in context

Taken as a whole, the thesis offers a process-level account of event-driven reliability. Its concern is the judgment that sits between an available mechanism and an adequate design choice: what must this process keep true, and what would it take to protect that condition or set of conditions under disturbance? That question places the argument in a wider conversation about how distributed systems balance availability, coordination, latency, recovery, and semantic guarantees against the properties applications actually need.

### 6.2.1 From mechanism to process judgment

This places the thesis in direct relation to established work. End-to-end reasoning explains why a mechanism may be useful at one level and still be insufficient unless the final requirement is protected where it can actually be checked [4]. Work on coordination avoidance, CALM, and highly available transactions makes a related point from the side of coordination, availability, and semantic guarantees: stronger coordination or transactional support is justified by the property being protected [14, 15, 16]. Online event-processing work sharpens the same concern by showing that delivery, processing, and transactional guarantees remain bounded by what they actually cover, especially once external state or side-effecting systems are involved [17]. The framework carries that discipline into event-driven process design, where the question is how a handling choice bears on what the process must protect.

### 6.2.2 What the evidence is evidence of

The point becomes sharper in event-processing systems because signs of reliability are often indirect. A system may show activity, recovery, or convergence, but those signs do not settle the question on their own. They can mislead in both directions. What looks reassuring may not be enough, and what looks troubling may not matter. A correctness boundary draws those signs back into relation by asking whether they bear on the one thing that finally matters: whether the process still counts as acceptable.

The empirical work on event management in microservice architectures discussed earlier shows why this point matters in practice [1]. The study suggests that event-based systems remain difficult to reason about at the level where design and operation meet. Developers have to decide how events should be produced and consumed, how event flows can be inspected, how dependencies between events should be handled, how ordering assumptions affect behaviour, and how failures can be debugged after the work has moved across service boundaries. These are practical difficulties, and they also point to a deeper reasoning problem: the system may expose many traces of what happened while still leaving unclear what those traces mean for the application process.

The boundary-first view is useful at precisely that point. It asks what the evidence is evidence of. Did the process still protect what made it acceptable? Did the disturbance matter for what had to hold? Did the recovery restore what had to be restored, or only produce activity that looked like recovery? The framework returns the problem to first principles by grounding event-driven complexity in a process-level judgment: what must still hold for the process to count as acceptable?

### **6.2.3 Contribution and claim boundary**

The empirical suite supports this perspective in a controlled form. Its setting is narrow by design, but that narrowness lets the ambiguity become visible: the same handling behaviour can carry different meanings depending on the condition at stake.

Seen in that light, the thesis is a small contribution to a larger software-engineering concern: ensuring that distributed work remains accountable to the outcome it is meant to produce. Event-driven architectures make that concern acute because execution can become easier to trace than process acceptability is to discern. The framework and conceptual outlook developed here give that judgment firmer process-level footing.

## 6.3 Conclusion

Reliability in event-driven processes comes into clearer focus once what makes a process acceptable has been made explicit. Without that requirement, delivery, recovery, and settlement can too easily be mistaken for reliability in themselves. With it, a handling model can better be judged by what it actually contributes to the process outcome.

This thesis developed a boundary-first framework for making that requirement and its implications explicit. A correctness boundary marks the acceptability boundary for an event-driven process, defined by one protected condition or by a connected set of protected conditions. The controlled empirical suite used deadline-constrained, state-oriented, and required-effect processes to show how the same handling models can carry different meanings under disturbance. Retention and replay may preserve recoverable work, arrive too late to matter, become unnecessary when later state overtakes a missed update, or fail to address producer omission altogether. Their value depends on the process requirement they serve.

The contribution is modest but practical: a process-level way to relate correctness boundaries, failure modes, handling choices, and trade-offs. The guide gives that reasoning an applicable form, and the central idea is simple: clarity about what must hold provides clarity about how it should be handled.

---

## References

- [1] R. Laigner, A. C. Almeida, W. K. G. Assunção, and Y. Zhou, “An empirical study on challenges of event management in microservice architectures,” *ACM Trans. Softw. Eng. Methodol.*, advance online publication, 2025, doi: 10.1145/3776581. [Online]. Available: arXiv PDF. Accessed: May 8, 2026.
- [2] R. Laigner, Y. Zhou, M. A. V. Salles, Y. Liu, and M. Kalinowski, “Data management in microservices: State of the practice, challenges, and research directions,” *Proc. VLDB Endow.*, vol. 14, no. 13, pp. 3348–3361, 2021, doi: 10.14778/3484224.3484232. [Online]. Available: PVLDB PDF. Accessed: May 8, 2026.
- [3] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, “The Dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proc. VLDB Endow.*, vol. 8, no. 12, pp. 1792–1803, 2015, doi: 10.14778/2824032.2824076. [Online]. Available: Google PDF. Accessed: May 8, 2026.
- [4] J. H. Saltzer, D. P. Reed, and D. D. Clark, “End-to-end arguments in system design,” *ACM Trans. Comput. Syst.*, vol. 2, no. 4, pp. 277–288, 1984, doi: 10.1145/357401.357402. [Online]. Available: MIT PDF. Accessed: May 8, 2026.
- [5] Y. Saito and M. Shapiro, “Optimistic replication,” *ACM Comput. Surv.*, vol. 37, no. 1, pp. 42–81, 2005, doi: 10.1145/1057977.1057980. [Online]. Available: HAL PDF. Accessed: May 8, 2026.
- [6] H. Garcia-Molina and K. Salem, “Sagas,” in *Proc. 1987 ACM SIGMOD Int. Conf. Management of Data*, 1987, pp. 249–259, doi: 10.1145/38713.38742. [Online]. Available: Princeton PDF. Accessed: May 8, 2026.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, “The many faces of publish/subscribe,” *ACM Comput. Surv.*, vol. 35, no. 2, pp. 114–131, 2003, doi: 10.1145/857076.857078. [Online]. Available: Columbia PDF. Accessed: May 8, 2026.
- [8] D. Patterson, A. Brown, P. Broadwell, G. Candea, M. Chen, J. Cutler, P. Enriquez, A. Fox, E. Kiciman, M. Merzbacher, D. Oppenheimer, N. Sastry, W. Tetzlaff, J. Traupman, and N. Treuhaft, “Recovery Oriented Computing (ROC): Motivation, definition, techniques, and case studies,” Univ. California, Berkeley, Berkeley, CA, USA, Tech. Rep. UCB/CSD-02-1175, Mar. 2002. [Online]. Avail-

- able: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2002/CSD-02-1175.pdf>. Accessed: May 8, 2026.
- [9] J. Dean and L. A. Barroso, “The tail at scale,” *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794. [Online]. Available: author PDF. Accessed: May 8, 2026.
  - [10] M. Fragkoulis, P. Carbone, V. Kalavri, and A. Katsifodimos, “A survey on the evolution of stream processing systems,” *VLDB J.*, vol. 33, no. 2, pp. 507–541, 2024, doi: 10.1007/s00778-023-00819-8. [Online]. Available: Springer PDF. Accessed: May 8, 2026.
  - [11] P. Helland, “Idempotence is not a medical condition,” *Queue*, vol. 10, no. 4, pp. 30–46, 2012, doi: 10.1145/2181796.2187821. [Online]. Available: ACM Queue PDF. Accessed: May 8, 2026.
  - [12] B. Alpern and F. B. Schneider, “Defining liveness,” *Inf. Process. Lett.*, vol. 21, no. 4, pp. 181–185, 1985, doi: 10.1016/0020-0190(85)90056-0. [Online]. Available: Cornell PDF. Accessed: May 8, 2026.
  - [13] Amazon Web Services, “Transactional outbox pattern,” *AWS Prescriptive Guidance*. [Online]. Available: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html>. Accessed: May 8, 2026.
  - [14] P. Bailis, A. Fekete, M. J. Franklin, A. Ghodsi, J. M. Hellerstein, and I. Stoica, “Coordination avoidance in database systems,” *Proc. VLDB Endow.*, vol. 8, no. 3, pp. 185–196, 2014, doi: 10.14778/2735508.2735509. [Online]. Available: PVLDB PDF. Accessed: May 8, 2026.
  - [15] J. M. Hellerstein and P. Alvaro, “Keeping CALM: When distributed consistency is easy,” *Commun. ACM*, vol. 63, no. 9, pp. 72–81, 2020, doi: 10.1145/3369736. [Online]. Available: arXiv PDF. Accessed: May 8, 2026.
  - [16] P. Bailis, A. Davidson, A. Fekete, A. Ghodsi, J. M. Hellerstein, and I. Stoica, “Highly available transactions: Virtues and limitations,” *Proc. VLDB Endow.*, vol. 7, no. 3, pp. 181–192, 2013, doi: 10.14778/2732232.2732237. [Online]. Available: PVLDB PDF. Accessed: May 8, 2026.
  - [17] M. Kleppmann, A. R. Beresford, and B. Svingen, “Online event processing: Achieving consistency where distributed transactions have failed,” *Commun. ACM*, vol. 62, no. 5, pp. 43–49, 2019, doi: 10.1145/3312527. [Online]. Available: author PDF. Accessed: May 8, 2026.
  - [18] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *J. Manage. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302. [Online]. Available: institutional PDF. Accessed: May 8, 2026.
  - [19] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Q.*, vol. 28, no. 1, pp. 75–105, 2004, doi: 10.2307/25148625. [Online]. Available: VUB PDF. Accessed: May 8, 2026.

- [20] Docker Inc., “Docker Compose,” *Docker Docs*. [Online]. Available: <https://docs.docker.com/compose/>. Accessed: May 8, 2026.
- [21] Redis Ltd., “Redis Pub/Sub,” *Redis Docs*. [Online]. Available: <https://redis.io/docs/latest/develop/pubsub/>. Accessed: May 8, 2026.
- [22] Redis Ltd., “Redis Streams,” *Redis Docs*. [Online]. Available: <https://redis.io/docs/latest/develop/data-types/streams/>. Accessed: May 8, 2026.
- [23] A. Vogel, S. Henning, E. Perez-Wohlfeil, O. Ertl, and R. Rabiser, “A comprehensive benchmarking analysis of fault recovery in stream processing frameworks,” in *Proc. 18th ACM Int. Conf. Distributed and Event-Based Systems*, 2024, pp. 171–182, doi: 10.1145/3629104.3666040. [Online]. Available: arXiv PDF. Accessed: May 8, 2026.
- [24] J. Tahir, R. Mayer, C. Doblander, and H.-A. Jacobsen, “How reliable are streams? End-to-end processing-guarantee validation and performance benchmarking of stream processing systems,” *Proc. VLDB Endow.*, vol. 18, no. 3, pp. 585–598, 2024, doi: 10.14778/3712221.3712227. [Online]. Available: PVLDB PDF. Accessed: May 8, 2026.
- [25] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Proc. 13th Int. Symp. Stabilization, Safety, and Security of Distributed Systems*, 2011, pp. 386–400, doi: 10.1007/978-3-642-24550-3\_29. [Online]. Available: author PDF. Accessed: May 8, 2026.

# Appendix A. Boundary-first design guide

Table A.1: Boundary-first guide for process-handling choice.

| Design check                         | Process-level question                                                                                               | What the answer indicates                                                                                                                                                                                                                                         | Handling implication                                                                                                                                                                                                     |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scope the process                    | Which specific event-driven process is being judged, and where does it begin and end?                                | The unit of judgment is the scoped process. A larger application may contain a connected chain of such processes.                                                                                                                                                 | If several conditions appear, state whether they define one connected boundary or belong to separate follow-on processes with their own boundaries.                                                                      |
| State the boundary                   | What must hold for this process outcome to count as acceptable?                                                      | The relevant protected condition or connected set of protected conditions is made explicit.                                                                                                                                                                       | Do not choose replay, retention, immediate handling, deferred settlement, or supporting safeguards before the correctness boundary is clear.                                                                             |
| Check boundary coherence             | Can the stated condition or connected set of conditions all hold within this scoped process?                         | Connected conditions are joint requirements. If they cannot hold together, this indicates a boundary-coherence problem: the process has been scoped around requirements that either belong to separate connected processes or cannot hold together as formulated. | Clarify the process boundary, or split the larger flow into connected processes, before choosing a handling model. Replay, retention, and settlement cannot make incompatible requirements hold together.                |
| Identify how the boundary could fail | What kind of failure would make the stated condition or conditions fail?                                             | The relevant failure mode may be delay, backlog, missed handling, stale state, repeated action, settlement delay, or source omission.                                                                                                                             | Mechanisms should be judged against the boundary-failure mode they can actually address.                                                                                                                                 |
| Check deadline sensitivity           | Would completion still count if it arrived late?                                                                     | Timeliness is part of correctness, not merely a performance concern.                                                                                                                                                                                              | Retention and replay help only if preserved work can still complete before expiry. Use immediate or deadline-aware handling when lateness invalidates the case.                                                          |
| Check state relevance                | Can later state overtake a missed or delayed update?                                                                 | Replay matters when the missed update remains relevant to the visible or resulting state. It matters less where the representation can stay acceptably close to canonical state by other means.                                                                   | Direct reads or bounded refresh may be sufficient where the representation can remain acceptably close to canonical state. Retained/immediate is stronger where missed state can remain stale or harmful.                |
| Check downstream recovery            | Was the event emitted, but missed or delayed downstream?                                                             | The failure is inside the consumer-side handling path.                                                                                                                                                                                                            | Retained/immediate can recover emitted work through replay when the downstream effect still has to happen.                                                                                                               |
| Check duplicate harm                 | Would repeated execution of the downstream action be harmful?                                                        | Correctness may require the effect to happen without harmful repeated execution.                                                                                                                                                                                  | Retained/deferred may be appropriate when duplicate effects are costly and later settlement is acceptable.                                                                                                               |
| Check settlement tolerance           | Is deferred settlement acceptable for this process?                                                                  | Deferred settlement can reduce duplicate execution, but may add reconciliation work and settlement delay.                                                                                                                                                         | Use retained/deferred only when the cost of later settlement is preferable to the cost of harmful duplicate effects.                                                                                                     |
| Check source omission                | Could the event fail to enter the flow at all?                                                                       | The failure lies before downstream handling. Replay and retention cannot recover what was never emitted.                                                                                                                                                          | Treat this as a producer-integrity concern before applying the downstream process-handling guide.                                                                                                                        |
| Choose the handling model            | Which handling model is proportionate to the stated condition or conditions, given the failure mode being addressed? | The model should fit the condition or conditions at stake, not appear strong in the abstract.                                                                                                                                                                     | Choose transient/immediate, retained/immediate, or retained/deferred according to what the process must protect. Record any necessary supporting safeguard, the residual risk, and the metric that would reveal failure. |

## Appendix B. Empirical suite synopsis

---

The empirical suite is a controlled event-processing environment used to examine how the process-handling models behave under selected protected conditions and disturbances. Its purpose is to make a small number of handling differences observable: whether work can still complete before deadline expiry, whether missed state remains relevant, whether emitted work can be recovered through replay, whether source omission leaves downstream replay with nothing to act on, and whether deferred settlement shifts cost into later reconciliation.

Each scenario cell combines three elements: a condition module, a disturbance profile, and a process-handling model. The condition modules correspond to the three kinds of protected conditions used in the thesis: deadline-constrained, state-oriented, and required-effect. The process-handling models are transient/immediate, retained/immediate, and retained/deferred, except in the state-oriented module, where the comparison focuses on transient/immediate and retained/immediate because duplicate containment and deferred settlement are not the state-oriented contrast being examined.

A run begins with a producer emitting the configured workload into the configured event flow. Redis Pub/Sub is used for transient publication, while Redis Streams is used where emitted events must remain available for replay. The configured disturbance then affects the run: processing may pause or slow down, work may accumulate, emitted events may be missed downstream, source omission may prevent some intended cases from being emitted as events, or repeated triggers may create duplicate-effect pressure. The selected handling model processes, preserves, replays, or settles the work according to the scenario.

Reporting scripts collect both endpoint outcomes and runtime traces. Deadline-constrained scenarios report measures such as deadline miss rate, completion-in-time rate, unresolved burden, doomed burden, repayment pattern, and resolution pulse. State-oriented scenarios report latest-state attainment and outage-time exposure. Required-effect scenarios report attainment, received-event counts, duplicate side effects, reconciliation work, and settlement delay. Each scenario cell was repeated ten times, giving 34 cells and 340 completed runs.

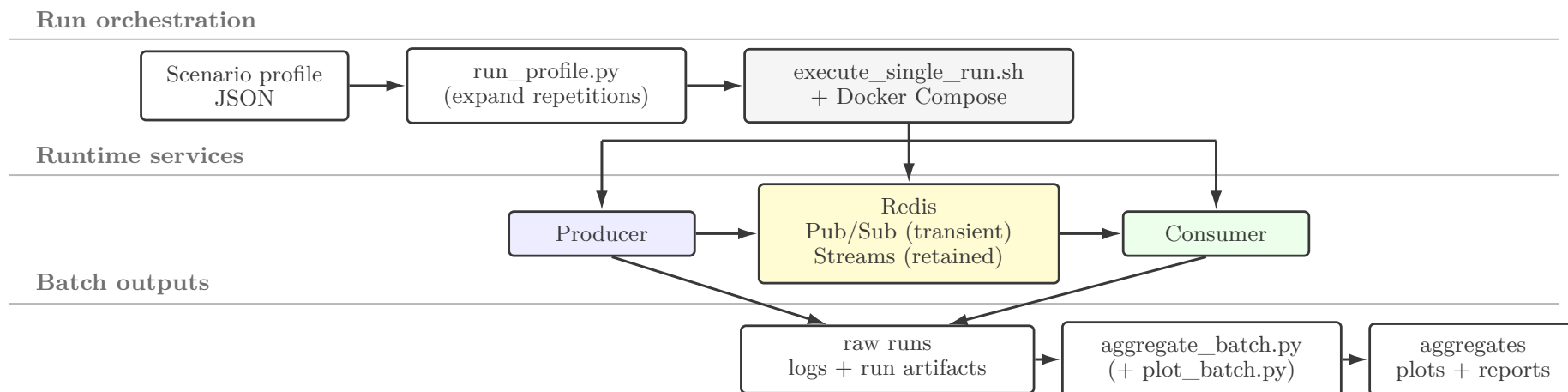
Figure B.1 gives a compact overview of this run flow. Tables B.1–B.4 give the operational definitions, scenario parameters, disturbance-channel mapping, and evaluation controls used in the reported suite. The reported values should be read as controlled comparisons within this research setting, not as quantitative predictions for deployed event-driven systems.

### Host-machine context

The reported runs were executed on a local Linux Mint 21.2 Cinnamon machine using Linux kernel 5.15.0-168-generic, an Intel Core i3-2348M CPU at 2.30 GHz and 8 GiB memory. These details are included as reproducibility context. Given the modest scale of the suite and the limited load it places on the host machine, the host specifications are not treated as variables in the comparison.

# Empirical Suite Execution Pipeline

Canonical run flow from scenario profile to aggregated outputs



**Figure B.1:** Empirical suite execution pipeline from scenario profile to aggregated outputs.

**Suite run stability.** Across repeated runs, the suite shows stable principal comparative patterns. For the selected outcome measures, mean and median values remain close, which suggests that the repeated distributions are generally not strongly skewed. Variation appears mainly in timing-sensitive deadline cases, where small shifts in scheduling or recovery timing can move an outcome from just before to just after the deadline boundary. Even there, the variation does not affect the suite’s principal patterns, which remain stable across repeated full runs. State-oriented and required-effect measures varied less, especially where the reported outcome concerns final state attainment, missing effects, duplicate effects, or eventual settlement. This is consistent with the suite’s intended role as a pattern-illuminating empirical setup rather than a precision benchmark. The empirical chapters therefore rest on a frozen, stable canonical output set, while the runnable suite remains available for reruns that can be compared against those outputs.

Table B.1: Operational definitions of reported metrics.

| Metric                                        | Computation rule                                                                                                | Used for             | Interpretation                                                               |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------|------------------------------------------------------------------------------|
| Deadline miss rate                            | Expired cases divided by produced cases.                                                                        | Deadline-constrained | Share of cases that expired before deadline-valid completion.                |
| Completion-in-time rate                       | Cases completed before expiry divided by produced cases.                                                        | Deadline-constrained | Share of cases that reached a valid outcome in time.                         |
| Unresolved burden                             | Produced cases not yet attained at a runtime sample.                                                            | Deadline-constrained | Work still unsettled during the run.                                         |
| Doomed burden                                 | Unsettled produced cases whose deadline has already passed at a runtime sample.                                 | Deadline-constrained | Work that remains unresolved after timely correctness is no longer possible. |
| Repayment pattern                             | Reduction of unresolved burden before cases cross the deadline, read with completion-in-time and doomed burden. | Deadline-constrained | Shows whether accumulated work can still be worked down in time.             |
| Resolution pulse                              | Completions and expiries counted per fixed runtime bin.                                                         | Deadline-constrained | Burstiness of resolution activity during recovery.                           |
| Required-effect attainment                    | Cases whose required downstream effect occurred divided by produced cases.                                      | Required-effect      | Share of required effects that happened.                                     |
| Received-event count                          | Events observed by the consumer after omission or duplicate pressure.                                           | Required-effect      | Separates downstream loss from source omission and duplicate triggering.     |
| Duplicate side-effect count                   | Repeated downstream executions beyond the intended effect.                                                      | Required-effect      | Measures harmful duplicate execution.                                        |
| Correction rewrites and reconciliation passes | Deferred overwrites and settlement sweeps recorded by the retained/deferred model.                              | Required-effect      | Measures the cost introduced by duplicate containment.                       |
| Settlement delay                              | Time from producer completion to last recorded attainment.                                                      | Required-effect      | Shows whether the retained/deferred model shifts cost into later settlement. |
| Latest-state attainment rate                  | Cases whose resulting latest state is acceptable divided by produced cases.                                     | State-oriented       | Measures final state adequacy.                                               |
| Outage-exposed seen fraction                  | Seen outage-relevant updates divided by expected outage-relevant updates.                                       | State-oriented       | Separates final adequacy from outage-time exposure.                          |

Table B.2: Scenario and parameter specification.

| Module               | Scenario family                   | Cells | Workload                                                                           | Boundary parameter                              | Disturbance specification                                                                                        | Handling models |
|----------------------|-----------------------------------|-------|------------------------------------------------------------------------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------|
| Deadline-constrained | Baseline                          | 3     | 1000 cases, 5 ms emission interval.                                                | 1.2 s deadline window, 30 s observation window. | No disturbance.                                                                                                  | T/I, R/I, R/D.  |
| Deadline-constrained | Backlog shock                     | 3     | 1000 cases, 5 ms base emission interval.                                           | 1.2 s deadline window, 35 s observation window. | 2.8 s interruption triggered at 20% progress, with overload emission interval of 1 ms for 6 s starting at 1.5 s. | T/I, R/I, R/D.  |
| Deadline-constrained | Moderate degradation              | 3     | 1000 cases, 5 ms emission interval.                                                | 1.2 s deadline window, 35 s observation window. | Sustained 8 ms processing delay over 8.0 s.                                                                      | T/I, R/I, R/D.  |
| Deadline-constrained | High degradation                  | 3     | 1000 cases, 5 ms emission interval.                                                | 1.2 s deadline window, 35 s observation window. | Sustained 50 ms processing delay over 8.0 s.                                                                     | T/I, R/I, R/D.  |
| Required-effect      | Baseline                          | 3     | 1000 cases, 5 ms emission interval.                                                | Required effect, no deadline window.            | No disturbance.                                                                                                  | T/I, R/I, R/D.  |
| Required-effect      | Handling gap, standard            | 3     | 1000 cases, 5 ms emission interval.                                                | Required effect, no deadline window.            | 3.5 s handling interruption starting at 2.0 s.                                                                   | T/I, R/I, R/D.  |
| Required-effect      | Handling gap, extreme             | 3     | 1000 cases, 5 ms emission interval.                                                | Required effect, no deadline window.            | 5.0 s handling interruption starting at 1.5 s.                                                                   | T/I, R/I, R/D.  |
| Required-effect      | Source omission                   | 3     | 1000 cases, 5 ms emission interval.                                                | Required effect, no deadline window.            | 20% of intended cases omitted before emission, leaving 800 emitted cases.                                        | T/I, R/I, R/D.  |
| Required-effect      | Duplicate pressure, standard      | 3     | 1000 cases, 5 ms emission interval.                                                | Required effect, no deadline window.            | 40% duplicate-selected cases, two duplicate repeats, 1 ms duplicate delay, 2 ms jitter.                          | T/I, R/I, R/D.  |
| Required-effect      | Duplicate pressure, extreme       | 3     | 1000 cases, 5 ms emission interval.                                                | Required effect, no deadline window.            | 70% duplicate-selected cases, two duplicate repeats, 1 ms duplicate delay, 2 ms jitter.                          | T/I, R/I, R/D.  |
| State-oriented       | Backlog shock                     | 2     | 300 cases $\times$ 3 updates (900 emitted updates), 0.3 obsolete replay fraction.  | Latest-state adequacy after final update.       | 2.4 s interruption triggered at 80% progress.                                                                    | T/I, R/I.       |
| State-oriented       | Backlog shock with forward resume | 2     | 300 cases $\times$ 4 updates (1200 emitted updates), 0.3 obsolete replay fraction. | Latest-state adequacy after final update.       | 0.8 s interruption triggered at 60% progress, followed by forward resumption.                                    | T/I, R/I.       |

Table B.3: Disturbance-to-channel map.

| Scenario family     | Primary disturbance locus                                | Observable effect                                          | Replay expectation                                                           |
|---------------------|----------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------------|
| Handling gap        | Downstream handling interruption.                        | Events are emitted, but missed or delayed by the consumer. | High for retained modes when the event remains available.                    |
| Source omission     | Producer-side non-emission.                              | Some intended cases are never emitted as events.           | None. Replay cannot recover an event that was never emitted.                 |
| Backlog shock       | Temporary processing interruption and catch-up pressure. | Work accumulates and then resolves in bursts.              | Depends on whether the remaining validity or relevance window is still open. |
| Degradation         | Sustained processing slowdown.                           | Work progresses under continuous latency pressure.         | Limited by deadline windows and accumulated doomed burden.                   |
| Duplicate pressure  | Repeated triggering of the same obligation.              | Duplicate downstream execution risk increases.             | The retained/deferred model can absorb repeated triggers before settlement.  |
| State backlog shock | Outage over a state-relevant update window.              | State may be unseen, lost, or restored by later updates.   | High when the missed state remains relevant to correctness.                  |

Table B.4: Evaluation controls and scope protection.

| Control objective      | Operational check                                                                                                   | Evidence in the suite                     | Residual limitation                                                |
|------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------------------------------|
| Pattern stability      | Each scenario cell was repeated ten times.                                                                          | 34 cells and 340 completed runs.          | Single-suite setting, not an external validity claim.              |
| Channel separation     | Source omission and handling gaps are separate scenario families.                                                   | Required-effect module.                   | Does not model every producer or broker failure mode.              |
| Boundary discipline    | Deadline, state, and required-effect modules are reported separately.                                               | Module structure and metric groups.       | Larger applications may contain several process-level obligations. |
| Traceability           | Raw runs, run specifications, metric summaries, aggregate outputs, and plots are preserved in the project artifact. | Batch folder structure.                   | Reproduction still depends on the local Docker/Redis runtime.      |
| Interpretive restraint | Results are read as controlled comparisons.                                                                         | Method, Results, and Limitations wording. | No claim of platform benchmarking or universal quantitative law.   |

## Supplementary material

---

Two supplementary resources accompany the thesis. The companion site gives a brief introduction to the thesis, provides access to the thesis PDF, links to selected resources for further reading on event-driven systems, and points to the empirical suite repository. The empirical suite repository contains the frozen canonical result set used for the thesis, including raw runs, aggregate summaries and material for rerunning and comparing the suite.

- Companion site: <https://kh3rm.github.io/thesis-companion-site/>
- Empirical suite page: <https://github.com/kh3rm/thesis-empirical-suite>

The thesis text remains the authoritative account of the argument, method, results, and interpretation.